DA User Guide

DA (short for *Digital Archive*) puts your photos and scanned documents in order. You copy them into one folder, and DA:

- sets exact duplicates aside, so you keep one copy of each photo,
- works out when each photo was taken, and renames it by that date (`photo_19980704_101500_ab12.jpg` was taken on 4 July 1998 at 10:15:00),
- files it in a folder for its year and month (`photos\organized\1998\1998-07\`).

DA never deletes anything. It asks before it moves anything, and it keeps a record of every change, so every step can be checked and most can be undone.

DA runs on Windows 10 and 11.

Contents

- Words you'll see
- Getting started
- The menu
- Common workflows
- Less common workflows
- Location and GPS workflows
- Config tokens
- If something goes wrong
- Reference

Words you'll see

Word	What it means
Archive	A folder DA looks after, for example `D:\jane-archive-199x`. You can have as many as you like, for example one for each person and each decade.
Intake	The folder inside an archive where you put new photos and documents: `intake\`. DA takes them from here.
Organized	Where DA files them: `photos\organized\` and `documents\organized\`, one folder per year and month.
Duplicate	An exact copy of a photo, byte for byte. DA keeps one and moves the others to `_duplicates-holding\`.
Sidecar	A small note DA keeps next to each file, named like the file plus `.meta.json`. It holds the date DA found, a fingerprint of the file, and any tags. Leave sidecars where they are.
Terminal	A window where you type commands. Windows calls it *PowerShell* or *Terminal*. DA's menu runs in one.
Command	Something you type in the terminal, like `da status`, then press **Enter**.

Getting started

1. Get DA

- **Download** DA.exe (the latest version), or get the file from whoever set DA up for you (for example on a USB stick).
- Your browser may ask whether to keep the file. Choose **Keep**.

2. Install it (once per computer)

1. Open **File Explorer** and go to the folder with `DA.exe` (usually **Downloads**).
2. Click in the **address bar** at the top of the window, type `powershell`, and press **Enter**. A terminal window opens in that folder.
3. Type this and press **Enter**: `powershell .\DA.exe install`
4. If Windows shows "*Windows protected your PC*", click **More info**, then **Run anyway**.
5. If it says **Long paths are off** and asks *Turn long paths on now?*, press **Enter** (yes), then click **Yes** when Windows asks for permission. See Long paths for why.
6. **Close the terminal window.** From now on, `da` works in any new terminal window.

DA is now at `C:\Scripts\DA\DA.exe`. To update DA later, download the new `DA.exe` and do these steps again; your archives aren't touched.

Long paths

Windows normally refuses a file path of **260 characters or more** (the folder names plus the file name). Files copied from deep folders, like `intake\docs\from-old-pc\users\jane\school\2022\course\...`, can get that long, and setting copies aside (into `_duplicates-holding\<date>\...`) adds about 30 more. Windows has a setting that lifts the limit, **long paths**, and DA works with it.

- `da install` checks it and offers to turn it on. Windows asks for permission (you need to be an administrator of the computer). It takes effect in **new** windows.
- While it's off, DA **skips** a file whose new path would be too long, carries on with the rest, and says so: *path too long for Windows*. Turn it on and run again; the skipped files are done then.
- To turn it on by hand: right-click **Terminal** (or **Windows PowerShell**), choose **Run as administrator**, and type: `powershell New-ItemProperty -Path HKLM:\SYSTEM\CurrentControlSet\Control\FileSystem -Name LongPathsEnabled -Value 1 -PropertyType DWORD -Force`

Windows won't run DA.exe? If you see "*an application control policy has blocked this file*", or Windows Security keeps warning about `DA.exe`, your computer has **Smart App Control** turned on. It only runs programs that are digitally signed, and DA.exe isn't. Don't turn your protection off: install DA with Python instead. It takes about ten minutes, once.

Install DA with Python (if Windows blocks DA.exe)

What's Python? A free, widely used program that runs programs written in the Python language, which DA is. Python itself is signed by the people who make it, so Smart App Control lets it run, and Python runs DA. You don't need to learn any Python: you install it once, type three lines, and then use DA exactly as described in this guide.

A. Install Python (once per computer)

1. Go to python.org, point at **Downloads**, and click the yellow **Download Python** button. Your browser saves the installer (usually in **Downloads**).
2. Open it (double-click it, or click it in the browser's downloads list).
3. Follow what it shows:
   - If you see a window with an **Add python.exe to PATH** box at the bottom, tick it, then click **Install Now**.
   - If a black window asks questions instead (the newer **Python install manager**), type **y** and press **Enter** each time, until it says it's done.
4. When it's finished, close the installer.
5. **Check it:** open a new terminal (press the **Windows** key, type `powershell`, press **Enter**), type this and press **Enter**: `powershell py --version` It answers with something like `Python 3.14.7`. If it says `py` isn't recognized, close the terminal, open a new one, and try again; if it still isn't, run the installer again.

B. Install DA (once per computer)

In a terminal, type these three lines, pressing **Enter** after each, and wait for each to finish (the second one downloads DA and the pieces it needs; it takes a minute or two and prints a lot):

```
py -m venv C:\Scripts\DA\python
C:\Scripts\DA\python\Scripts\python.exe -m pip install -r https://da.gamenightbymike.com/python/r
C:\Scripts\DA\python\Scripts\python.exe -m mindit install
```

What they do: 1. Make a private copy of Python just for DA, in `C:\Scripts\DA\python`. Nothing else on your computer is changed. 2. Download DA (from this website) and what it needs into that copy. 3. Set up the `da` command and update every archive's `Open DA.cmd` to use it. (If an old `DA.exe` is in `C:\Scripts\DA`, it's renamed `DA.exe.off`, so it doesn't get in the way.) It also offers to turn on long paths.

Close the terminal and open a new one. Type `da version`: it shows DA's version. From here on, everything in this guide works the same: `da`, `Open DA.cmd`, the menu.

C. Update DA later: type `da update`. It downloads the newest version and says which one you have now. If it says it couldn't finish, close every DA window and run it again.

3. Create your first archive (once per archive)

1. Open a new terminal: press the **Windows key**, type `terminal`, press **Enter**.
2. Type this (with your own drive, name and years) and press **Enter**: `powershell da setup D:\jane-archive-199x --years 1990-1999` **Years** can be a range (`1990-1999`), one year (`--years 1999`) or a decade (`--years 199x`); leave it out for any year.
3. **Already in the folder?** `cd` there and use `--here` (add `--yes` to skip the question): `powershell cd D:\jane-archive-1999 da setup --here --yes --years 1999` Or just type `da` there: the menu says you're not in an archive and offers **H** - *make this folder an archive*. DA won't make an archive of a drive's top folder, Windows' folders or your user folder itself.

What happens depends on the folder:

- **A new folder:** DA creates it, with everything inside it that an archive needs.
- **A folder you've already been copying photos into:** DA lists what's there and asks "*Make it an archive and move these N items into intake\?*". Type **y**. Everything moves into `intake\` with its names unchanged, and the move is written down so it can be undone.
- **A folder that's already an archive:** DA asks "*Use the existing archive?*", keeps its settings, and adds anything missing.

`--years 1990-1999` is optional. With it, a photo taken in another decade (say 2005) is left in `intake\` and listed for you, so you can move it to the right archive (see da split).

Your archive now looks like this:

```
D:\jane-archive-199x\
  Open DA.cmd           <- double-click this to open DA in this archive
  da-archive.yaml       <- this archive's settings
  intake\
    photos\             <- put photos here (any subfolders, any names)
    docs\               <- documents: PDF, Word, Excel
    scans\              <- what your scanner saves: PDFs and scanned pictures
  photos\organized\     <- DA files photos here, by year and month
  documents\organized\  <- and documents here
  output\               <- DA's records and reports
```

4. Open it

Double-click `Open DA.cmd` in the archive folder. DA's menu opens. That's all you need from here on.

The menu

```
DA - jane-archive-199x   D:\jane-archive-199x   years: 1990-1999
intake: 1,204 files waiting    organized: 5,873 photos, 12 documents    last run: 2026-10-13 21:
Run includes steps: 1,2,3,4,5,6,7,9   (P to change)

 intake: 1,204 files (from the index)
      1,180  phone-2024
         24  scans
 last run: 2026-10-13T21:31:05  filed 5,873

 1 Autorun    P Pre-run setup    2 Status    3 What needs review    4 Undo last set-aside
 5 Refresh index     6 Switch archive    7 New archive
 L Location folders  S Split by years    M Merge from another copy    R Report (PDF)
 B Dashboard (browser)    D Fix dates    V Date photos (review)    C Cameras
 N Naming    J Update sidecars    U User guide    K Back up    H Help    Q Quit
```

The menu opens with the archive's **Status**: what's waiting in intake, folder by folder, and the last run. It shows it again at the end of **1 Autorun**, and when you switch to another archive. Type the number or letter, then press **Enter**.

Key	What it does
1 Autorun	Files what's in `intake\`, doing the steps the pre-run setup chose. See da run.
P Pre-run setup	Chooses which steps **1 Autorun** does and whether it asks before each change; kept for next time. See Choose the steps.
2 Status	This archive at a glance: what's waiting, what's filed, the last run.
3 What needs review	What the last run left for you to look at, with the reasons.
4 Undo last set-aside	Puts the duplicates from the last run back where they were.
5 Refresh index	Updates DA's list of files, for example after you moved files yourself.
6 Switch archive	Open another archive on this computer.
7 New archive	Lists the archives this computer knows, then creates a new one (or turns a folder into one).
L Location folders	Groups a month's GPS photos into folders by place. See Group a month into location folders.
S Split by years	Shows which files in intake belong to another decade's archive, and moves them there. See da split.
M Merge from another copy	Copies in what another copy of this archive has and this one doesn't. See Merge two copies of an archive.
R Report (PDF)	Makes the one-page report and opens it.
B Dashboard (browser)	Opens the archive's dashboard in your web browser, and goes straight back to the menu.
D Fix dates	Give a date to photos that have none (a folder at a time), then file them. See Fix dates.
V Date photos (review)	Lists the photos waiting for a date in the review folder, then opens the dashboard's **Dates** page to date them. See Scanned prints.
C Cameras	Lists every camera and scanner the archive has seen (`cameras.json`), and changes one's owner, type, date rule or years. See Cameras.
N Naming	Shows how files are named and foldered, tries new patterns, and renames and moves the filed files to match (with undo). See Change how files are named.
J Update sidecars	Brings older sidecars (the `.meta.json` files) up to date. See Update sidecars.
U User guide	Opens this guide in your web browser.
K Back up	Copies every archive on this computer to the backup drive. See Back up an archive.

Key	What it does
H Help	A short reminder of how DA works.
Q Quit	Leaves the menu. The window stays open for typing commands.

Typing `da` on its own in any terminal opens the menu too. Outside an archive, it lists the archives on this computer for you to pick from.

Common workflows

Workflow	What it's for	Menu
Add photos from a phone or camera	Get photos into intake	(File Explorer)
Add scanned documents	Get paper into the archive	(your scanner)
da import	Bring in a whole drive, only what isn't archived yet	`da import`
da run	File what's in intake	**1**
da report	See what's there and what needs you	**2**, **3**
Fix what needs review	Help DA with photos it couldn't date	**3**
Duplicates: put back or delete	Undo a set-aside, or free the space	**4**
Edit a filed photo	Crop or brighten a photo and keep the original	(your photo editor)
da split	Move photos from another decade to the right archive	`da split`
Contact sheets	See a whole folder of photos on one page	(made for you)
da find	Search by month, name or tag	steps below
da tag	Label files (`beach` , `grandma` , `taxes`)	steps below
Back up	Copy every archive to the backup drive	**K**
Switch between archives	Work in another archive	**6**
Update DA	Get the newest version	steps below

`da run` is a command you can type. The other "da ..." names stand for a few commands or menu items used together.

Add photos from a phone, camera or memory card

1. **Open the archive's intake folder** in File Explorer: `D:\jane-archive-199x\intake\photos\`.
2. **Make a folder for this batch,** named so you'll recognise it, e.g. `phone-2024-summer`.
3. **Copy the photos into it:**
   - **iPhone:** connect it with its cable and unlock it. Tap **Trust** if it asks. In File Explorer, open **This PC > Apple iPhone > Internal Storage > DCIM** and copy the folders you want. (See iPhone HEIC photos about the photo format.)
   - **Android phone:** connect it, choose **File transfer** on the phone, then open **This PC > (your phone) > Internal storage > DCIM > Camera**.
   - **Camera or memory card:** put the card in the computer (or connect the camera), then open its drive and the **DCIM** folder.
4. **Copy, don't move,** until DA has filed them and you've checked the result.
5. Then process them.

Any folder names and file names are fine. DA works out the dates itself.

Add scanned documents

1. **Set your scanner to save into** `D:\jane-archive-199x\intake\scans\` (or copy the files there afterwards).
2. **PDF is best** for documents. Pictures (`.jpg` , `.png` , `.tif`) saved in `scans\` are treated as documents too, not as photos.
3. Word and Excel files and other PDFs go in `intake\docs\`.
4. Then process them. Documents are filed in `documents\organized\` by the date inside them (when they were scanned or created).

da import: bring in a whole drive or folder

Years of photos and papers often sit on several drives, many of them copies of each other: a `Pictures` folder, an old backup, a USB drive. `da import` brings in only what the archive doesn't have yet, sorted for you, and never changes the drive it reads from.

```
da import E:\Pictures                          # a preview: what would be copied, and why
da import E:\Pictures F:\old-backup --apply   # copy them (asks first)
```

Run it inside the archive that takes the files (open it with **Open DA.cmd**, or `cd` into it). For each file on the drive, DA:

1. **Skips what's already in an archive.** It checks every archive on this computer by the file's content, so a photo that's already filed, or already waiting in intake, isn't copied again. A file met twice on the drives (the same photo in `Pictures` and in an old backup) is copied once, from the first folder you named, so name the most complete one first.
2. **Sorts it:**

    - a photo from a camera or phone, a video, a RAW or HEIC file goes to `intake\photos`;
    - a PDF with text, a Word or Excel file, or a scan that Windows reads as a page of writing goes to `intake\docs`;
    - any other picture (a scanned print, an old photo with no camera details) goes to `intake\scans`.
3. **Dates papers.** A document whose name has no date gets the first date written on its first page, so a bill files under the month it was sent.
4. **Copies it** into `intake\<docs, photos or scans>\from-<drive and folder>\`, keeping the folders it was in. Folder names like `2019-04-20 Easter` become tags and the name offered for its location folder.

Without `--apply` nothing is copied: you see how many files are new, already in an archive, or copies, and where the new ones would go. Every file is listed, with the reason, in `output\runs\<time>-import\import-plan.csv`. With `--apply` DA shows the same and asks **Copy them?** first. Then choose **1 Autorun** to file them, and **S Split by years** for files that belong to another decade's archive.

Good to know: - Files DA doesn't keep (music, programs, `.psd` files ...) are counted and left where they are, as are `Thumbs.db`, the Recycle Bin and other system folders. - `--exclude <folder>` leaves a folder out; `--no-ocr` skips reading pictures (pictures with no camera details then go to `intake\scans`). - `da intake undo "<the import-log.csv it names>"` removes the copies again. - When a second run of `da import` on a drive says **Nothing to copy**, every file there is in an archive. - The archive's `intake:` settings need `scans: photo` (new archives have it). If not, `da import` says what to change in da-archive.yaml.

Keep track of several drives: da imports

With years of folders going into several archives, a plan and a progress page keep track of it all.

1. **Make the plan once**, from inside any archive: `powershell da imports --init E:\Pictures F:\old-backup` It writes `imports.yaml` in the folder that holds your archives: each year folder (`E:\Pictures\2015`) goes to the archive for its year, and everything else on the drive to the newest archive. Open it in Notepad to check it, or to add more drives (`source:` and `archive:` lines).
2. **Bring in an archive's share:** in that archive, `da import --plan --apply` copies every folder the plan gives it that hasn't been brought in yet. Then **1 Autorun**, and **S Split by years** if it says so.
3. **See how far it's got:** `da imports` prints a table and writes `imports.html` beside the plan. Open it in your browser and leave it open: it updates itself after every import, Autorun and Split, and reloads every minute. For each archive it shows what's been copied, filed, set aside as a copy, still waiting, or needing a date, and the **Next** thing to do there.

da run: file new photos and documents

Menu: 1 Autorun. DA lists the folders in `intake\` and how many files each holds:

```
intake\photos\
 #  folder                          files
 0  process all of intake\photos\  5,081
 1  phone-2024-summer\             1,204
 2  scans-from-grandma\              877
  number: open   0: process   text: filter   .. up   Enter: back
```

- Type a **number** to open that folder. A folder with no folders inside it is processed straight away.
- Type **0** to process the folder you're in, everything in it.
- Type some **text** (e.g. `grandma`) to show only the folders whose names contain it.
- Type **..** to go up a level, or just press **Enter** to go back.

DA says which steps it will do (`Run includes steps: 1,2,3,4,5,6,7,9`, as chosen in the pre-run setup) and works through them. Before each step that changes anything, it asks; type **y** to go ahead, or **n** to stop there (nothing after that point changes). With **Bypass yes** on in the setup it doesn't ask.

Step	What happens	Changes files?
1 Index	Lists everything in the archive	No
2 Hash	Fingerprints the photos that could be copies	No
3 Duplicates	Moves a Mac's hidden `._` files (see below), exact copies, then resized copies (the same picture, smaller), to `_duplicates-holding\`	**Yes, if you say y**
4 Preview	Works out each photo's date, new name and folder	Only writes the sidecar notes
5 Check	Shows how many will be filed, by year, and warns about anything odd	No
6 File	Renames and moves the photos into `organized` as the preview planned, then checks each one arrived intact (in place, same size and date, a sample re-read in full), and makes each folder's contact sheet	**Yes, if you say y**
7 Refresh	Updates the index	No
8 Camera owners *(if on)*	Asks who used each camera it hasn't seen before (Enter skips one); see da tag	Only `da-archive.yaml` (`cameras:`)
9 Tags *(on at first)*	Automatic tags for the photos just filed: camera, owner, year, place, people	Only the sidecar notes
10 Places *(if on)*	Looks up the town and state of the GPS photos just filed (OpenStreetMap, about one new spot a second; nothing is sent if `location: lookup: off`)	Only the sidecar notes
11 Location folders *(if on)*	For each month filed into that has GPS photos: shows its groups and asks a title for each, as da location group does	**Yes, after you name them**
12 Faces *(if on)*	Finds the faces in the photos just filed and suggests names (da facial); you confirm them on the dashboard's People page	Only DA's index

Steps 8-12 work on the folders this run filed into, not the whole archive. A step that is off says *skipped*. **From a terminal:** `da run` does the steps you chose in the menu (at first: tags); `--skip` and `--with` change that for one run, by name or number:

```
da run --skip tags                    # just steps 1-7 this time
da run --with places,groups           # look up places and make location folders too
da run --yes --with 10                # no questions; places too
```

With `--yes`, nobody is there to answer, so steps 8 and 11 only say what's waiting (*2 cameras with no owner yet - da tag --cameras*; *2019-04 ... da location group 2019-04*).

How long it takes: a few minutes for a few hundred photos; roughly an hour for every 50,000. It's fine to leave it running.

What stays in intake, on purpose: - **Photos with no date anywhere** (none inside the file, none in its name). See Fix what needs review. - **Photos from another decade** (with `years` set). See da split. - **Videos** (`.mov`, `.mp4`): DA doesn't file videos yet. The video half of an iPhone Live Photo is the exception: it's filed with its photo.

Camera raw files (Canon `.CR2` and `.CR3`, Nikon `.NEF`, Sony `.ARW`, `.DNG` ...): - **A raw file with its JPG** (`IMG_0574.CR3` beside `IMG_0574.JPG`, as a camera set to *RAW + JPEG* makes them) is filed right next to the JPG, under the same new name: `photo_..._1f40.jpg` and `photo_..._1f40.cr3`. The contact sheet shows the picture once. - **A raw file on its own** is filed as a photo: by the date and camera in it, named like any photo (`photo_20130803_165246_9fb9.cr2`). Its contact sheet tile shows the preview the camera stored in it, and `da tag` reads its camera like any photo's. - Raw files aren't compared as copies of other pictures (their built-in preview isn't the full picture).

Running it again is always safe. Files already filed aren't touched, and new ones are processed the same way.

From a terminal (handy for leaving a big run going overnight):

```
da run                                  # all of intake
da run intake\photos\phone-2024-summer  # one folder
da run --yes                            # don't ask; go straight through
da run E:\old-backup                    # a folder outside the archive: DA offers to copy it in f
```

Choose the steps: P Pre-run setup

Menu: P Pre-run setup lists every step, ticked when **1 Autorun** does it:

```
Pre-run setup: what 1 Autorun does each time   up/down: move   space: switch   Enter: keep   Es
File what's in intake:
  [*]  1  Index             (always) what the archive and this folder hold now
  [*]  2  Hash              (always) fingerprint the files that could be copies
> [x]  3  Duplicates        set exact, resized and re-saved copies aside (off: copies are filed
  [*]  4  Preview           (always) plan every name and folder (nothing moves)
  [*]  5  Check             (always) show the plan and its warnings
  [*]  6  File              (always) move the files into organized and check each one
  [x]  7  Refresh index     index again at the end (off: the next run catches up)
After filing (on the folders just filed):
  [ ]  8  Camera owners     ask who used cameras DA hasn't seen yet
  [x]  9  Tags              automatic tags for the photos just filed
  [ ] 10  Places            look up the places of GPS photos (internet, about one a second)
  [ ] 11  Location folders  group each month's GPS photos into folders you name
  [ ] 12  Faces             find the faces in the photos just filed and suggest names (da facia
Questions:
  [ ]     Bypass yes        don't ask before each change (8 and 11 then only say what's waiting
```

- **Up** and **down** arrows move the `>`; the **spacebar** switches the step it points at on or off.
- Steps marked `[*]` are always done: filing needs them. **3** and **7** can be left out, and **8-12** are up to you.
- **Bypass yes** answers *yes* to every question, so a run goes start to finish on its own. Camera owners (8) and location folders (11) need you, so with it on they only say what's waiting.
- **Enter** keeps your choice, **Esc** (or **q**) leaves it as it was.

What you choose is kept in da-archive.yaml under `run:` (`after_filing:`, plus `skip:` and `bypass: true` when you set them), and the menu's blue box shows it each time (`Run includes steps: 1,2,4,5,6,7,8,9,bypass`). `da run` follows the same `after_filing:` and `skip:`, but still asks before changes unless you add `--yes`.

da report: what's in the archive and what needs you

Menu: 2 Status and **3 What needs review**, or `da report` for a page to print or keep.

A one-page report: type `da report` (add `--open` to open it straight away). DA makes a dated PDF, `output\jane-archive-199x-report-2026-10-14.pdf`, with: - what's filed (photos, documents, space used, the years they span), and how many by year - what's waiting for you (intake, duplicates set aside, notes to review) and the last run - the cameras used, with photo counts, and who took them (from da tag) - the places: how many photos have GPS, a sentence like "*This archive includes photos taken in 2 countries (United States, France) and 9 US states (Illinois, Florida, ...)*", and the most photographed towns (from da location lookup)

The numbers come from DA's index. If you moved files yourself, use `da report --refresh`. A second report on the same day replaces the first.

1. **2 Status:** the archive's name and years, what's waiting in intake, what's filed, the last run.
2. **3 What needs review:** what the last run left behind, grouped by reason, with examples:
    - *No date (in the file or its name)*
    - *Taken outside this archive's years*
    - *File name and camera date disagree* (DA used the date in the name)
    - *Duplicate inside the folder* (left for you to decide)

    The full list is saved as `review.csv` in that run's folder (DA shows where). It opens in Excel. 3. **More detail, from a terminal:** `powershell da db stats # how many photos and documents, by year da intake status # how far each intake folder has got da validate photos\organized # check what's already filed: names, folders, notes (nothing moves)` 4. **Every run's record** is kept in `output\runs\<date-time>-run\` : a summary, the duplicate list, and the preview and filing reports, file by file.

If you added or moved files yourself, choose **5 Refresh index** first so the numbers are current.

da dashboard: the archive in your browser

Menu: B Dashboard (browser), or type `da dashboard` .

DA opens a page in your web browser showing the archive as it is now, and keeps it up to date every few seconds. It's handy in a second window while `da run` works: - **Filed:** photos, documents, videos and the space they use - **Waiting in intake:** how many files, folder by folder - **Needs review:** what the last run left for you, by reason (**3 What needs review** lists the files) - **Happening now:** a DA command still working in this archive (in another window, say) - **Latest runs** and **Activity:** what DA did, newest first - **In the archive:** photos by year, the cameras and the places. Counting these takes a while in a very big archive, so DA counts them once when the page opens; **Recount** counts them again. - **Pictures:** the latest filed, or any month (**Show**), as small pictures. Click one to see it large: photos (iPhone HEIC photos too), a PDF in the browser's own viewer, a video played in the page (most MP4 and MOV videos; some iPhone HEVC videos only where the browser can play them). Left and right arrow keys go through the month; **Esc** closes it. Under the large picture: - **Set as desktop:** makes the photo this computer's desktop background. - **Open its folder:** opens File Explorer with the file selected. - **Share:** emails the photo to one of your share lists. - **People** (**Show**, or open the page at `...#people`): the faces da facial found, to name them - see there.

Apart from its buttons, the page doesn't change your files, and only this computer can open it (its address starts `http://127.0.0.1:`). It only shows what's filed in `organized` ; documents set aside by da find --passwords never show.

It keeps running in the background, so your terminal (or the menu) is free straight away, and closing the terminal doesn't stop it. Close the browser tab whenever you like; `da dashboard` (or **B**) again opens the same dashboard, it doesn't start a second one. To stop it:

```
da dashboard --stop
```

Restarting the computer stops it too. To run it in the terminal window instead (until **Ctrl+C**), use `da dashboard --foreground`.

The numbers come from DA's index, like the report's: after files were moved by hand, choose **5 Refresh index**.

Fix what needs review

Choose **3 What needs review**, then for each reason:

- **No date:** DA couldn't find when the photo was taken. If you know, even roughly, give it one: see Fix dates (menu **D**).
- **Outside this archive's years:** it belongs in another archive. See da split.
- **Name and camera date disagree:** usually the camera's clock was wrong. DA used the date in the name. If that's the wrong one, set the right date (or shift the camera's clock): see Fix dates.
- **Duplicate inside the folder:** two identical files. DA leaves both; delete the one you don't want.

Fix dates: photos with no date, or the wrong one

Scans of old prints, photos from a camera whose clock was never set, files that lost their date when they were copied: DA can't tell when they were taken, so they wait - in the review folder (`photos\organized\review`, see Scanned prints), or in intake in an archive made before November 2026. You can tell DA the date. A date you give is kept with the photo (in its sidecar) and always wins over the camera and the file name.

The easy way (menu D Fix dates): 1. Choose **D**. DA lists each intake folder that has photos with no date, one at a time: `For each folder, type the date its photos were taken: 1998, 1998-07 or 1998-07-04. scans\album-1 (48, e.g. scan0001.jpg): 1998-07 scans\album-2 (31, e.g. scan0049.jpg): 1999 phone-old (12, e.g. IMG_0003.JPG):` 2. Type a date for a folder, press **Enter** to skip one, or **q** to stop. 3. DA asks whether to process intake now. Say yes, and they're filed by the dates you gave.

Dates you can type:

You know	Type	Filed under
The day	`1998-07-04`	`1998\1998-07\photo_19980704_000000_xxxx.jpg`
The day and time	`1998-07-04 14:30`	`1998\1998-07\photo_19980704_143000_xxxx.jpg`
The month	`1998-07`	`1998\1998-07\` (as the 1st)
Only the year	`1998`	`1998\1998-01\` (January); a month is better if you know it. A scanned print waits in review for a month
Only the decade	`1990s`	Not filed: noted, and it waits in review

From a terminal (also for photos already filed): - See which photos have no date: `da date list` (or `da date list intake\scans`). - Give a folder (or photos) a date: `da date set intake\scans\album-1 --date 1998-07`. - **A camera's clock was off** (an hour after a time change, or a year that was never set): shift all its photos by the same amount: `da date shift intake\phone-old --by +1h` (or `-1y`, `+2d`, `-30m`, `+3mo`). One amount at a time; each shift adds to the last, so *8 years and 4 months back* is `--by -8y`, then `--by -4mo`. - **A camera whose clock was reset** (its battery ran out) dates the rest of a trip years later. When some photos in a folder are filed and others there are dated outside the archive's years, **Process intake** says so at step 5: "*501 photos in ...\trip are dated 2017, while 327 beside them are dated 2008 - a camera clock that was reset?*" Those stay in intake. Move them into a folder of their own, find one whose real date you know, and shift the folder by the difference; then process intake again. - Changed your mind? `da date clear <photos or folder>`: the photo's own date counts again.

For photos already in `organized`, DA then says how to move them to match: `da validate "<their folder>" --mode repair` renames and moves them by the new date.

Scanned prints: the review folder and the Dates page

A scanner writes **the day it scanned** as the picture's date. A 1975 print scanned in 2022 says 2022, so DA never takes a scan's own date as the day it was taken. Archives made since November 2026 say so in `da-archive.yaml` (`scans: scanphoto` under `intake:`); `da import` puts a scanner's pictures in `intake\scans`. A picture from a camera listed as a `scanner` in cameras.json counts as a scan wherever it is.

Where a scan's date comes from: 1. A date you set (the Dates page, `da date set`, `da run --date`). 2. A date someone typed into the picture before (Windows' *Date taken*): it's earlier than the day the file was saved, or older than home scanners. 3. Its name: `Scan 1987-03.jpg`. 4. The folders it came in from: `from-D-1998\06-15 Birthday party` is **1998-06-15**; `from-D-1995\Christmas` is only **1995**. Folders that name the scanning (`2023-04-22 scanned`, `Box 3`, `check dates`) are skipped, and so is any date from the year it was scanned or later.

A month is enough to file it (`1998\1998-06`). A year alone, or nothing, and it waits.

The review folder. Autorun moves photos with no sure date out of intake into `photos\organized\review\<the folders they came in from>`, with their names unchanged. Intake then holds only what hasn't been looked at, and the photos to date are together. Once a photo there has a date, the next **1 Autorun** (or `da review --file`) files it like any other. If the date belongs to another decade, it stays in review until **S Split** (`da split --organized`) moves it to that archive.

The Dates page (menu **V**, or the dashboard's **Dates** card):

```
Folder [from-D-Pictures\Box 3 (412)]  [-] [+]  [Select all]  [File dated photos]   Last set: ...

 [x] picture   [x] picture   [ ] picture   [ ] picture
 guess 1987-03  guess 1987-03
 Sam, Jane     Sam

 [--] [-]  Older    1987-03    Newer  [+] [++]    [Set]  [Edit]
 best guess: 1987-03 (folder: 03-14 Lake trip)
```

1. Pick a folder, then click the photos that go together (**Shift**-click a run; **Select all**). `-` and `+` make the pictures smaller or bigger; double-click one to see it big.
2. The big month starts at the best guess. **−** and **+** move it a month, **−−** and **++** a year (the arrow keys do too, with **Shift** for a year). Click the month to type it.
3. **Set** gives the selected photos that month. **Undo** takes the last one back.
4. **Edit** (one photo): its exact date, the camera it was **taken with** (a film camera from cameras.json: its owner and years then count for the photo), people and tags.
5. **File dated photos** files them now, as Autorun would.

From a terminal: `da review` lists the folders waiting, with the best guess for each; `da review --guess` keeps each photo's guesses in its sidecar; `da run intake\scans\box-3 --date 1987-03` dates a whole box as it's filed (Autorun asks for the same when you pick one folder).

Cameras: who owned each, and how to read its dates

`cameras.json` (in the archive, or one shared by a family's archives: `cameras_file: ..\cameras.json` in `da-archive.yaml`) lists every camera and scanner. Menu **C**, or:

```
da cameras --detect                       # add the ones in the photos, and count them
da cameras                                # the list
da cameras set "Nikon D70" --owner sam --years 2004-2009
da cameras set "Nikon D70" --date "shift +1h"        # its clock was an hour off
da cameras add "Pentax K1000" --owner jane --years 1985-1990    # a film camera (its prints are s
```

Column	Means
type	`camera` , `phone` , `film` or `scanner`
owner	Who it belonged to: the `owner:` tag (it replaces `cameras:` in `da-archive.yaml`)
dates	`exif` trust its clock; `scanned` its date is when it scanned; `shift +1y` its clock was off by that much; `none`
years	When it was used: a film camera's years are the guess for a scan said to be from it

Duplicates: put back or delete

Exact copies are moved to `_duplicates-holding\<date>\` in the archive, in the same folders they came from.

- **Put them back: 4 Undo last set-aside**. Or, for an earlier run, from a terminal: `da intake undo "<run folder>\set-aside-moved.csv"` .
- **Delete them** to free the space, once you're happy with the result. Check first that each one's original is still there, then let DA empty it: `powershell da holding # checks every copy; nothing changes da holding --everywhere # also looks for originals in your other archives da holding --empty # sends the copies whose original was found to the Recycle Bin (asks first)`
    - **The check** matches each copy to the file it's a copy of: an exact copy by its fingerprint (SHA-256), a resized or re-saved one by the original DA noted when it set it aside (followed through any renames and moves since). It reads a sample of the originals again, to be sure they haven't changed.
    - DA says how many have their original and how many **stay** (*no original*, *couldn't be read*, *original changed*), and lists every file in `output\runs\<date-time>-holding\holding-check.csv` .
    - **Emptying** removes only the copies whose original was found, with their `.meta.json` ; anything else stays where it is, and folders left empty go. If they're too big for the Recycle Bin, Windows asks before deleting them for good: answer **No** to keep them. `holding-emptied.csv` lists what went.
    - Or delete the whole `_duplicates-holding` folder yourself: every file DA put in it is an exact copy of one that was kept, or (in its `resized` and `resaved` folders) another copy of a picture that was kept.

Copies that aren't exact: resized, re-saved, edited, cropped

Programs like Picasa and Photoshop Elements made other copies of photos: smaller ones to email (`IMG_0100-1.JPG` , 800 x 600, beside the camera's 3072 x 2304 `IMG_0100.JPG`), the same picture saved again (often losing its date), edits and crops. They are the same picture but not the same file, so the exact-copy check can't see them. DA compares the **pictures** themselves (a small fingerprint of each, worked out once and kept, and the pixels of any close pair):

Kind	What DA does
Resized copy (the same picture, smaller)	Set aside in `_duplicates-holding\<date>\resized\` ; the largest stays
Re-saved copy (the same pixels, saved again)	Set aside in `_duplicates-holding\<date>\resaved\` ; the one with the camera's date stays
Edited version (a photo editor changed its look)	Both stay filed, linked to each other
Cropped version (a part of the photo, from the same moment)	Both stay filed, linked to each other
A photographer's export (Lightroom / Photoshop: `For Print` , `For Social Media` ...)	Kept, every size: never set aside as a copy. Linked to the RAW it was made from and to its other exports (the export names its RAW inside it). `da edits` lists them; `--apply` writes the links (`da run` does it for the folder it processes)
A Mac's hidden `._` file (`._IMG_7102.CR2` , a few KB)	Set aside in `_duplicates-holding\<date>\` : a Mac leaves one beside every file it copies to a Windows drive. It holds the Mac's own notes about the file, not a picture, so it has no date and would otherwise stay in intake forever. DA checks its contents, not just the name

Two shots of the same scene a moment apart are never one of these: DA checks the camera's time and the pixels.

- **Processing intake** does it at step 3, right after exact copies: the resized and re-saved copies move to holding, and the one kept is filed. A sample of the plan is checked again first, and you're asked before anything moves.
- **For photos already filed:** `powershell da similar # shows what it found; nothing changes da similar --apply # sets the copies aside (asks first) and links the versions` The first time, DA looks at every photo (a few minutes for 5,000); after that only new ones.
- Picasa faces, your own tags, and the keywords and title Photoshop Elements wrote into a copy go to the one kept (the keywords become your tags, so you can find them).
- **Linked versions** get a `versions` block in their sidecars: which is the original and which are edited or cropped, found again by fingerprint after a rename or move. `versions.csv` in the run folder lists them.
- `similar-moved.csv` (or `resized-moved.csv` in a run) lists each one; `da intake undo "<that file>"` puts them back.
- `da history --duplicates` lists them with the exact copies, and da share shows them.
- **Bursts and blurry photos** are marked by `da similar --apply` , never moved:
  - A **burst** is shots of the same thing taken one after another (each within 2 minutes of the one before, and alike). Each gets the tag `burst` , and its sharpest shot `burst:best` , so you can keep the best and let the rest go: `da findit tags photos\organized -r burst -v` .
  - A **blurry** photo is far softer than that camera's other photos: out of focus, or taken by accident in a pocket. It gets the tag `blurry` . Foggy or plain pictures that have a sharp part aren't marked. A camera needs 20 photos or more before DA judges its photos this way.
  - `bursts.csv` and `blurry.csv` in the run folder list them. The first time, working out how sharp each photo is takes a few minutes for 5,000 photos; after that only new ones.

Edit a filed photo: crop, brighten, fix red-eye

The rule: never change the filed photo itself. Save the edit as a new file in intake, and let DA file it next to the original. The original stays exactly as the camera made it, and the edit is filed and linked to it. If you don't like the edit later, the original is still there.

Why not just edit the filed photo? Saving it changes the file. Its fingerprint (SHA-256) no longer matches the one in its sidecar, so DA renames it, and the camera's original is gone for good.

Which program

Any photo editor works. These are good choices:

Program	Cost	Good for
Photos (comes with Windows)	Free	Crop, straighten, rotate, brighten, colour, red-eye, removing a spot. Enough for most photos.
Paint.NET or **GIMP**	Free	More: layers, cloning out a person or object, text.
Photoshop Elements or **Lightroom**	Paid	Many photos at once, old scans (dust, fading), camera raw files.
Digital Photo Professional (Canon), **darktable**	Free	Camera raw files (`.CR2` , `.CR3` , `.NEF`).

Avoid **Paint** (the simple one that comes with Windows). It throws away the camera details (camera, date, GPS) when it saves.

Common edits

- **Crop:** cut out the edges, or keep only one person. DA links a crop to its original too.
- **Straighten or rotate:** a tilted horizon, or a photo the camera saved sideways.
- **Brighten, contrast, colour:** a dark indoor photo, or an old print that has turned orange.
- **Red-eye** and **spot fixes:** dust on a scan, a blemish.
- **Black and white,** or a sepia look.

Step by step (Windows Photos)

1. **Find the photo** in `photos\organized\<year>\<year-month>\` , e.g. `photo_19980704_101500_ab12.jpg` . (da find or the dashboard help.)
2. **Open it** (right-click > **Open with** > **Photos**), then click **Edit** (the pencil).
3. **Make your changes.**
4. **Save it as a copy, not over the original:** click **Save options** > **Save as copy**.

    - **Where:** the archive's `intake\photos\` folder. A folder there of its own keeps edits together, e.g. `intake\photos\edits\`.
    - **Name:** the photo's own name with an ending, e.g. `photo_19980704_101500_ab12-edited.jpg` (`-crop`, `-bw`, `(2)` work too). The name tells DA which photo this is an edit of. It also keeps the date, even if the program drops the camera details.
5. **File it:** menu **1 Autorun** (or `da run`). The edit goes into the same month folder as the original, with a name of its own: the same date and time, and a new 4-letter ending (`photo_19980704_101500_7c3e.jpg`).
6. **Link the two:** `da similar --apply`. Both sidecars get a `versions` block naming the original and the edited one. Contact sheets show *+1 version (edited)*, and da share lists it. See Copies that aren't exact.

Other programs work the same way. Use **Save As** or **Export** (never plain **Save**), save into `intake\photos\`, and give it the photo's name plus an ending.

Good to know

- **Tags and faces stay with the original.** To tag the edit too: `da tag photos\organized\1998\1998-07\photo_19980704_101500_7c3e.jpg grandma`.
- **An edit that changed almost nothing** (opened and saved again) is a *re-saved copy*. Processing intake sets it aside in `_duplicates-holding\`, and the original stays filed.
- **A photo in a place folder** (`1998-07 Lake trip (Tahoe)`): the edit is filed in the month folder. `da location group 1998-07` offers to put it in the place folder too, if it kept its GPS position. You can also move it there with its `.meta.json`, then run **5 Refresh index**.
- **Picasa:** saving an edit in Picasa replaces the photo where it is and hides the original in a `.picasaoriginals` folder. Use **Export** to `intake\photos\` instead.
- **Lightroom and Photoshop Elements catalogs:** add photos to the catalog without moving them ("Add", not "Move" or "Copy"), and export edits to `intake\photos\`.
- **Don't want an edit any more?** Delete it and its `.meta.json` in File Explorer, then run **5 Refresh index** (`da db build`). DA itself never deletes anything.
- **If you did save over a filed photo:** the edit is filed under a new name the next time `organized` is checked (`da validate photos\organized --mode repair`). Then run **5 Refresh index**. The original can only come back from a backup.

da split: move files to the right decade's archive

An archive with `years` set (e.g. `1990-1999`) only files photos taken in those years. One taken in another decade stays in intake, listed under **3 What needs review** as "*Taken outside this archive's years*". `da split` moves those to the archive for their decade. You don't type the years: each archive's own `years` setting decides.

1. **See where they belong.** In the archive whose intake holds them (e.g. `D:\jane-archive-199x`), type `da split`. DA works out each file's date the same way `da run` does and says: `14 files in intake were taken outside 1990-1999. 11 belong in jane-archive-200x (2000-2009): da split --to "D:\jane-archive-200x" 3 from the 1980s have no archive on this PC yet. Make one: da setup <folder> --years 1980-1989`

2. **No archive for a decade yet?** Make one: `da setup D:\jane-archive-198x --years 1980-1989`.
3. **Move them:** `da split --to D:\jane-archive-200x`. DA lists what will move and asks first. Each photo goes with its sidecar, raw file and Live Photo video into the other archive's `intake\photos\from-jane-archive-199x\`, keeping the folders it was in. Documents go to its `intake\docs\`. A file whose name is already there stays put.
4. **File them there:** open the other archive (its `Open DA.cmd`, or **6 Switch archive**) and choose **1 Autorun**.

Good to know: - **Changed your mind?** `da split` prints an **Undo** line: `da intake undo "<the file it names>"` brings everything back. It works between drives too. - **Only some years:** `da split --to D:\jane-archive-200x --years 2000-2004`. - **An archive that takes any year** (no `years:`) has nothing "outside" it. To move some photos anyway, give `--years`. - Files with no sure date stay in intake. Fix their dates first (see Fix what needs review), then run `da split` again. - **Two archives taking the same years:** `da split` says so; the first one on this PC's list gets them unless you say which with `--to`.

Split filed files too: --organized

`da split --organized` does the same for photos, videos and documents **already filed** in `organized`. Use it to split a big archive that takes any year into the archives for each decade:

```
da split --organized                                        # see where they all belong (nothing move
da split --organized --to D:\jane-archive-200x          # move them (asks first)
da split --organized --to D:\jane-archive-200x --years 2000-2004    # a few years at a time
```

- DA finds them by the date in its index (it refreshes the index first), so it's quick even on a big archive.
- **Where they go:** when both archives name and folder files the same way (see **N Naming**), straight into the other archive's `organized`, in the same folders (location folders too). Otherwise, or when a file with that name is already there, into its `intake\photos\from-<this archive>\`, to be filed there with **1 Autorun**. DA says how many go each way.
- Each file goes with its sidecar, raw file and Live Photo video; folders it leaves empty are removed, and the contact sheets of both archives are made again (quickly: their thumbnails are kept). The other archive's index is updated straight away.
- Files with no date stay. **Undo:** the line DA prints; then **5 Refresh index** in both archives.

da share: send photos to family and friends

`da share` makes a simple contact sheet of one or more months and emails it **from your own Gmail** to a list of people: - **Page 1:** which archive they're from (its name and years), your note, then every picture small on retro **film strips**, 4 frames a strip, each with its number along the top and its short **ID** (`▶ 9B05`, from the picture's fingerprint) along the bottom. - **Page 1 also names the people** in the pictures (and how many each is in) and the most common tags. - **Then the pictures big,** 6 a page, with the date, place and ID under each, and a **yellow box** on each known face with the person's name above it (boxes only on the small film-strip frames). - **The email** says which archive they're from, starts with the people and tags, and lists every picture by the same ID: who is in it, its name, where it is in

your archive, its SHA-256 fingerprint, the camera, the GPS position and place, its tags, and when it was taken. So someone can say "*the one marked 7A60*" and you can find it. (The same list is kept as `details.txt` beside the PDF.)

```
da share photos\organized\2019\2019-04 --to family --note "Easter at grandma's"          # make i
da share photos\organized\2019\2019-04 --to family --note "Easter at grandma's" --send   # send i
```

- **The lists** are in da-archive.yaml; `da share --lists` shows them: `yaml share: lists: family: [grandma@example.com, sam@example.com] friends: [pat@example.com]` Or name people straight away: `--to "grandma@example.com, sam@example.com"`.
- **Each person gets their own email**, with only their address on it, so nobody sees who else got it.
- **Photos only:** videos and documents in those folders are left out (DA says how many). Documents set aside by da find --passwords are never in them.
- **Copies and versions** (exact, resized and re-saved copies in holding; edited and cropped versions still filed) are listed under each picture in the email (with *Burst: shot 3 of 8* ... and *Blurry* where they apply), and the PDF ends with a framed box per picture showing them - **Copies in holding** pages showing each one beside the picture kept. `--no-copies` leaves them out.
- **A big share** is sent as several emails (each under Gmail's size limit), each with its own film strips (the frame numbers carry on) and the list of its pictures.
- The email shows where photos are in your archive and their GPS positions: share with people you'd tell that to.
- The PDF is kept in `output\shares\<date-time>-<name>\`, so you can look at it first, or print it.
- **From the dashboard:** open a photo large and press **Share** to email just that photo to a list (with its details in the email).

Set up sharing (once)

DA sends through Gmail's own way for programs to send mail, and only ever asks for permission to **send**. You make a small "app" in Google's developer pages for DA, once (about ten minutes):

1. Open https://console.cloud.google.com and sign in with your Gmail account.
2. At the top, **Select a project** > **New project**. Give it a name of 4 to 30 characters (e.g. `DA-sharing`), then **Create**, and make sure it's selected.
3. **APIs & Services** > **Library**: search **Gmail API**, open it, **Enable**.
4. **APIs & Services** > **OAuth consent screen** (newer pages: **Google Auth Platform**): **External**, then fill in an app name (e.g. `DA-sharing`) and your email address where asked. Under **Test users** (or **Audience**), add your own Gmail address. Save.
5. **APIs & Services** > **Credentials** (or **Clients**) > **Create credentials** > **OAuth client ID**: application type **Desktop app**, any name (e.g. `Desktop client 1`), **Create**, then **Download JSON**.
6. Save that file as `%LOCALAPPDATA%\DA\google-client.json` (in File Explorer, type `%LOCALAPPDATA%\DA` in the address bar, then save it there with that name).
7. In a terminal: `da share --setup`. Your browser asks you to choose your account and allow DA to **send email on your behalf**. Google warns that the app isn't verified: it's your own app, so choose **Continue**.

- While the app is in *Testing*, Google asks you to sign in again every 7 days (`da share --setup`). To stop that, on the **OAuth consent screen** choose **Publish app** (it stays your own, unverified app).
- The sign-in is kept in `%LOCALAPPDATA%\DA\google-token.json` , never in an archive. Delete that file to sign out.
- **More computers:** one OAuth client is enough. Copy the same `google-client.json` into each computer's `%LOCALAPPDATA%\DA\` and run `da share --setup` there once; each keeps its own sign-in.
- **Someone else sending from their own Gmail** (on their computer): add their address as a test user (or publish the app), and they sign in with their own account in `da share --setup` .

da find --passwords: set aside documents that hold passwords

Old statements, scanned letters and notes sometimes hold a password or a PIN. `da find --passwords` finds them and moves them out of the archive, into `_secure-holding\` , to go into a password vault later.

```
da find --passwords                          # look (nothing moves): documents, intake, and photo
da find --passwords --move                   # move the ones that hold one (asks first)
da find --passwords documents\organized\2015 # just one folder
```

- **What it reads:** a PDF's own text, Word, Excel and PowerPoint files, text files, and scans (pictures, and the pages of a scanned PDF) with Windows' own text reader. Photos are only looked at when da tag ocr has read them already (a photo of a sticky note).
- **Holds one:** a label with a value, like `Password: ...` , `PIN 1234` , `passcode` , `pwd=` , `Security answer:` . These are moved with `--move` . **Only mentions one** ("*change your password*", a blank form line): listed; add `--include-weak` to move those too.
- **The list** is `passwords.csv` in `output\runs\<date-time>-passwords\` : each file and a snippet with the password **hidden** (`Password: ****`). DA never writes the password itself anywhere.
- **Moving** keeps each file's place (`_secure-holding\<date>\documents\organized\2015\...`), takes it out of DA's index (so the dashboard, reports and sharing never show it), and removes its read text from its sidecar and from DA's OCR notes. **Undo:** the line DA prints, then **5 Refresh index**.
- Reading scans takes a few seconds each; on a big archive, do a folder at a time.
- `_secure-holding\` is an ordinary folder: move its files into your password vault, then delete them.

da ocr: the dates in each document, and whether it holds a secret

`da ocr` reads each document once and notes two things in its sidecar: **every date written in it**, and **whether it holds a secret** (a password, a PIN) - Yes or No, never the secret itself. Document contact sheets show both.

```
da ocr                                  # every document in documents\organized (ones read before
da ocr documents\organized\2015         # one year (or month, or file)
da ocr --dates                          # only the dates (or --secrets: only secrets)
da ocr --again                          # read them all again
```

- **What it reads:** the same as da find --passwords: a PDF's own text, Word, Excel and PowerPoint files, text files, and scans with Windows' own text reader.
- **Dates:** every real day written out - `2009-01-04`, `1/4/2009`, `01-04-2009`, `1/4/09`, `January 4, 2009`, `4 Jan 2009`, `the 4th of January 2009` - from 1900 to next year. `2009-02-30` isn't a day, so it's left out. A document may have none, one, or many: the sidecar's `dates` block lists them, oldest first.
- **Secrets:** *Yes* when the text has a password, a PIN or a security answer (or only mentions a password: noted as weak), else *No*. The secret, and the text itself, aren't written anywhere. To move them out of the archive: da find --passwords --move.
- A document read before, and not changed since, is skipped, so running it again is quick.
- `ocr-facts.csv` in `output\runs\<date-time>-ocr-facts\` lists each document, how many dates (the first few), and Secrets yes/no.

da template: file one company's bills by what's printed on them

Bills and statements you download have names that say nothing (`download (3).pdf`), and the date inside the PDF is the day you downloaded it. A **template** tells DA how to recognise one company's documents and where the bill date, the account number, the amount and so on are printed. Then `da run` files each one by its bill date, with a name that says what it is (`acme-power_2025-09-15_4455_a709.pdf`), and keeps everything it read in the sidecar.

1. Make the template (once per company). Put two or three of the company's PDFs somewhere, then:

```
da template new acme-power --from D:\Downloads\acme1.pdf --from D:\Downloads\acme2.pdf
```

DA finds the company's name that every sample prints, and the usual fields: bill date, account number, name, service address, billing period, amount due, due date. It writes `templates\acme-power.yaml` in the archive and tries it on the samples, showing what it read from each one.

2. Check it, and change what it missed. `da template edit acme-power` opens it in Notepad:

```
title: "Acme Power"
match:
  all: ["Acme Power"]                    # printed on every one of these documents
  none: ["Acme Power Tools"]             # a look-alike's name: never one of these
fields:
  date:    {label: ["Bill date"], type: date, required: true}
  account: {label: ["Account number"], type: account, required: true, expect: ["910022334455"]}
  amount:  {label: ["Total amount due", "Amount due"], type: money, required: true}
  due_date: {label: ["Due date"], type: date}
file:
  name: "{template}_{date:%Y-%m-%d}_{account:-4}_{hash:4}"
  # folder: "Bills\{title}\{YYYY}"     # leave out: the archive's year\month folders
```

- **label** is the words printed just before the value. List more than one when the layout changed over the years; the more particular one goes first (*Total amount due* before *Amount due*).
- **type**: `date`, `money`, `account`, `text`, `lines` (an address), `period` (two dates: from - to), `number`.
- **required: true**: a document without it doesn't count as one of these. The date always is.
- **expect**: only these values count. Put your own account numbers here, so a bill for a neighbour or the house's last owner isn't taken for yours.
- **file: name** and **folder** use the fields as tokens: `{date:%Y-%m}`, `{account:-4}` (its last 4), `{amount}`, `{title}`, `{YYYY}`, `{hash:4}`.

Then try it on a folder holding the company's documents **and** some others; nothing is changed:

```
da template test acme-power D:\Downloads
```

Each document is *matched* (with everything read from it) or *failed* (with why). Change the template until every one of the company's documents matches and nothing else does. `da template list` shows the templates you have.

3. File them. Save the downloads into the archive's `intake\docs`, then:

```
da run --kind document --template acme-power intake\docs -y
```

- Each document is read first. One the template matches gets a line with what was read (*date 2025-09-15, account 910022334455, amount 1164.07* ...); then it's named, dated and filed like any document.
- Every other document stays where it is, with a line saying why: *gas bill.pdf read by acme-power template and failed: no 'Acme Power' in it*.
- `templates.csv` in the run's folder (`output\runs\<date-time>-run\`) lists every document: matched or failed, why, and every value read. The run record (`summary.json`) has the same list.
- More than one company at once: `--template acme-power --template city-water`, or `--template all`. A document two templates both match isn't filed (make one of them stricter).
- A scanned bill (a picture, not text) is read with Windows' own text reader, like da ocr.

- What was read stays in the document's sidecar (`template` block), and it gets the tags `template:acme-power` plus the template's own `tags:` , so da find finds them.
- To share templates between archives, add `templates: D:\templates` to each archive's da-archive.yaml.

Documents filed before you had the template. Run it on their organized folder instead of intake:

```
da template test acme-power documents\organized\2024          # first: which ones match (nothing
da run --template acme-power documents\organized\2024
```

Each one the template matches is renamed where it is (*3 filed already to rename where they are*), or, when the template has a `folder:` , moved there with its new name. A month folder left empty is removed, and the others' contact sheets are remade. Documents the template doesn't match aren't touched.

da find: search the archive

DA keeps an index of every file: its name, folder, date and tags. Choose **5 Refresh index** first if anything changed, then in a terminal in the archive:

- **Everything from one month** (by the folder it's filed in): `powershell da db query "SELECT file_name, folder FROM files WHERE area='photos' AND folder LIKE '2010\2010-07%'"`
- **By name:** `da db query "SELECT path, file_name FROM files WHERE file_name LIKE '%0704%'"` . To find a file by its original path or name after DA moved or renamed it, search `source_path` : `powershell da db query "SELECT path, area, folder, file_name, source_path FROM files WHERE source_path LIKE '%7a376dda4c8afe76-video.MOV%'"`
- **Documents:** use `area='documents'` instead of `area='photos'` . Other area values are `intake` and `holding` .
- **By indexed tags, date, and GPS:** `tags` is stored as a JSON list, so include the quotes around the tag when searching it with SQLite's `json_each` : `powershell da db query "SELECT f.path, f.captured_at, f.tags FROM files AS f, json_each(f.tags) AS tag WHERE lower(tag.value) = 'beach' AND f.gps_present = 1"`
- **By place and date range:** `captured_at` is usually ISO date/time text; `city` and `country_code` come from location metadata: `powershell da db query "SELECT path, captured_at, city, country_code FROM files WHERE city = 'Springfield' AND captured_at LIKE '2010-07%'"`

The index's main table is `files` . These are all its columns:

Column	What it contains
`id`	Internal row number.
`path`	File's current full path; use this to locate it now.
`area`	Indexed tree: `photos`, `documents`, `intake`, or `holding`.
`folder`	Folder relative to its area root (empty for the root itself).
`file_name`	Current filename, including extension.
`source_path`	Original path recorded in the sidecar, when available.
`module`	Metadata module, commonly `photoit` or `scanit`.
`media_type`	Kind of file: `photo`, `video`, `document`, `raw`, or `other`.
`extension`	Lowercase extension, such as `.jpg` or `.pdf`.
`size_bytes`	Current file size in bytes.
`modified_at`	Current file modification time.
`sha256`	Recorded SHA-256 content hash, when available.
`sidecar_path`	Full path to its `.meta.json` sidecar, when one was found.
`sidecar_schema_version`	Sidecar format version, when present.
`captured_at`	Capture/document date from metadata, or a date parsed from a DA filename.
`tags`	Tags from metadata as a JSON list, or `NULL` when there are no tags.
`gps_present`	`1` if GPS metadata was found, otherwise `0`.
`city`	City from location metadata, when available.
`state_code`	State or region code from location metadata, when available.
`country_code`	Country code from location metadata, when available.
`title`	Location-group title, when set.
`validation_status`	`ok` or `problems` after validation; `NULL` if not recorded.
`validation_problem_count`	Number of validation problems recorded.
`hash_status`	Hash/quarantine status from metadata, when available.
`duplicate_status`	Duplicate status from metadata, when available.
`indexed_at`	Time this row was last indexed.

Some values are `NULL` when the sidecar has no corresponding metadata or the file has not been through that workflow. `path`, `file_name`, and `folder` describe its current location; `source_path` is only as complete as the

original sidecar metadata. The index reflects its last build, so refresh it with **5 Refresh index** or `da db build` after files change. A build only reads the files that are new or changed since the last one (by size and modified time of the file and its sidecar, and DA's version), so it takes seconds when little has changed; `da db build --full` reads everything again. `da db query` accepts read-only `SELECT` queries; use `da findit tags` for tag searches when you want full tag matching rather than a SQL text search.

- **By tag or sidecar metadata**, use FindIt. Unlike `da db query`, FindIt reads the `.meta.json` sidecars in the folder you give it; it searches recursively by default. For example: ```powershell # Find files with the beach tag; -v shows the matching file paths and details. da findit tags photos\organized --require-tag beach --verbose

Require both tags; exclude anything tagged grandma.

da findit tags photos\organized -r beach -r family -x grandma -v

Find July 2010 photos that have GPS metadata.

da findit tags photos\organized --date-prefix 2010-07 --require-gps -v

Find tagged scanned documents, and ignore results whose files are missing.

da findit tags documents\organized -r receipt --module scanit --media-type document --file-exists -v ``Repeated -r `options mean **all** those tags must be present; repeated` -x `options exclude a match if it has **any** of those tags. Tag searches include your tags and DA's automatic tags, such as` owner:jane`,` year:2021`, and` gps`. Other useful filters are` --name-contains 0704 `(case-insensitive filename substring) and` --no-recursive` (only sidecars directly in that folder). Filters combine, so the result must satisfy every filter you give.

`-v` prints the matching files and details; without it, `tags` prints the search summary and match count. To save the full results as JSON, use `--output-file matches.json` or `--write-output` (which writes under `reports\findit`). - **See what metadata FindIt can read:** `da findit summary photos\organized` counts sidecars, files with and without tags, missing files, and metadata categories. It does not list individual matches. Add `--no-recursive` to limit the summary to that folder, or `--write-output` to save its JSON. The optional `--backend sqlite --db-path <database>` is only supported by `summary`; tag searches use the sidecar backend.

Or just browse: every photo is in `photos\organized\<year>\<year>-<month>\`, in date order, and each folder's `_contact-sheet.pdf` shows them all on a page or two.

Contact sheets: a folder at a glance

Every folder DA files photos into gets a **contact sheet**: `_contact-sheet.pdf`, small pictures of all its photos and videos. Double-click it to see a whole month at once, or print it.

- **Page 1 sums the folder up:** how many photos and videos, the days they span, how many have a GPS position, the places, which sidecar versions they have, and a table of the **cameras** - who used each, how many pictures, which days, and the page its pictures start on.
- **Then a section per camera.** Each page is headed with the folder and the camera, and who used it when DA knows (see da tag): `2018-04 Easter (springfield-il) - Apple iPhone 7 / annie`.
- **On each picture:** its name and the date and time it was taken underneath; **GPS** in green down its left edge if it has a position; and a small circled number at its top right: the version of its sidecar (`2` is current; see Update sidecars).
- **Faces:** a **yellow box** on each face DA knows - names you confirmed on the dashboard's People page and Picasa's (see Bring in Picasa's faces), with the person's name above it. Page 1 lists the **people** in the folder and how many pictures each is in.
- **Copies in holding:** when DA has set aside a copy of a photo, its tile says how many (*2 duplicates in holding*, or *3 copies in holding (1 resized, 1 re-saved)*); a photo with other versions still filed says *+2 versions (cropped)* or *cropped version - original filed too*; a burst says *burst 3 of 8* (*, best* on its sharpest), and a blurry photo *blurry*. Extra pages compare the primary with each held copy, with thumbnails, full file locations, sizes, and SHA-256 values.
- **12 pictures a page (3 across, 4 down)**, big enough to recognise. For more on a page, use 20 (4 x 5): `da contact-sheets <folder> --layout 4x5`, or for the whole archive set it in da-archive.yaml: `yaml contact_sheets: layout: 4x5`

- Making a sheet is quick: everything but the pictures comes from the sidecars, and each picture's small copy is kept in the archive's `.da\thumbs\` folder (about 22 KB each) under its fingerprint. So when a sheet is made again (after a rename, a move into new folders, or new copies in holding), the photos aren't opened again, which is about 3 times faster. You can delete `.da\thumbs\` at any time; DA makes them again as needed.

- **It keeps itself up to date.** When da run files more photos into a folder, its sheet is made again. If you add, remove or replace photos yourself, Check what's already filed (`da validate`) makes it again.

- **For folders filed before DA made sheets,** or any other folder of photos (even outside an archive), make them yourself; this does every folder inside the one you name: `powershell da contact-sheets photos\organized\1998` Add `--force` to make every sheet again. After `da tag` (new cameras or owners), a sheet notices and is made again the next time `da run` or `da validate` looks at its folder.
- **Click a picture to open the photo** (or video). The links are written from the sheet to the file beside it, without a drive letter, so they keep working when the archive is on another drive or another PC. Copies on the *copies in holding* pages open the same way. (Tried in Edge and Chrome; Windows asks which app to use for files the viewer can't show itself.)

- DA never files, indexes or checks the sheet itself, and deleting one does no harm.

Documents: one sheet per year. `--kind document` makes a sheet in each **year** folder of documents (`documents\organized\2009\_contact-sheet.pdf`), showing every document of that year - its month folders too - as a picture of its **first page**, 2 x 2 to a page (`--layout 3x3` for more):

```
da ocr documents\organized\2009                                  # first: the dates in each, and
da contact-sheets documents\organized --kind document            # a sheet in every year folder
da contact-sheets documents\organized\2009 --kind document       # just 2009
```

- **Page 1** sums the year up: the archive's name, documents and pages, how many `da ocr` has read, how many hold a secret, the dates found, copies in holding, sidecars, a line per month - and the **legend** of what's on each document.
- **On each document:** the number of **pages** in a square at the top left, the sidecar's **schema** version in a circle at the top right, and under it, all in one size: `File: doc_100102_234558_4dc5.pdf Folder: documents\organized\2010\2010-10 Dates: 1973-05-15, 2009-08-21, 2009-10-28, 2010-10-28` (*Dates: none found*, or the first few and *+N more*.) The last part of a DA name, `4dc5`, is the start of the file's SHA-256 fingerprint.
- **A red border:** `da ocr` found a secret in it (a password, PIN or security answer) - never shown on the sheet. **A blue border:** `da ocr` hasn't read it yet (or it changed since): run `da ocr` on the year, then make the sheet again. A document with copies in `_duplicates-holding` says *Holding: N duplicates*.
- A PDF shows its real first page; a scan, its picture. Word, Excel and PowerPoint files (.doc .docx .xls .xlsx .ppt .pptx) and text files show their first page too when the free **LibreOffice** (libreoffice.org) is installed: DA has it lay each one out once, in the background, and keeps the picture. Without LibreOffice they show the picture Office saved in the file, else a card with their first lines of text. Installed LibreOffice later? The next `da contact-sheets --kind document` remakes the sheets with the pages.
- **Click a first page, or its File line, to open the document** - from wherever the archive is, like photo sheets.
- Like photo sheets, it's only made again when something on it changed (a document, or what `da ocr` found).

da tag: label photos and documents

A tag is a word attached to a photo (`beach`, `grandma`, `trip:big-bear`), so you can find it later with da find. DA makes some tags for you, and you add your own.

Automatic tags. When DA first files a photo, it notes what the photo says about itself: the camera's maker and model, the lens, the camera's serial number, any owner name the camera recorded, the date and time, and the GPS position if the phone recorded one. From those, `da tag` makes:

Tag	Example	From
`maker:`	`maker:apple`	the camera's maker
`camera:`	`camera:apple-iphone-12-pro`	the camera's model
`owner:`	`owner:jane`	who owned that camera (see below)
`year:`	`year:2021`	when it was taken
`gps`	`gps`	the photo has a GPS position
`folder:`	`folder:jones-family-photos`	each folder it came in from (more)

1. **Make them for the whole archive:** in a terminal in the archive, `da tag`. It asks first, then works through every filed photo (photos filed before DA kept camera details are read once, roughly an hour per 50,000). To do one year or one folder: `da tag photos\organized\2021`.
2. **Say who owned each camera** (most cameras don't record it): `da tag --cameras` lists every camera found, with how many photos it took and in which years, then asks about each one with no owner yet: `Who used each camera? Type a name, or press Enter to skip. Apple iPhone 7 Plus (160 photos, 2019): jane Apple iPhone 8 (13 photos, 2019): Update the owner: tags now? [y/n] (y):` DA saves the names in the archive's da-archive.yaml for you (keeping a copy of the old file), and updates the `owner:` tags straight away. To set one without the questions: `da tag --owner "Apple iPhone 8" sam`. A camera listed with a serial number (`Nikon D90 #3012345`) is that one camera body; without one, every camera of that model.

Your own tags go on one photo, several, or every file in a folder (documents too). Name the files or folders first, then the tags:

```
da tag photos\organized\2021\2021-06 beach "big bear"
da tag photos\organized\2021\2021-06\photo_20210601_100000_ab12.heic grandma
da tag photos\organized\2021\2021-06 --remove beach
```

Tags are kept in lower case, with spaces as `-` (`big-bear`). Making the automatic tags again never changes yours.

Pictures with text in them (receipts, screenshots, photos of letters or forms): `da tag ocr` reads the text in each picture of a folder, using Windows' own text reader (nothing to install). It sorts them in three: - **document:** mostly text (20 words or more, covering a good part of the picture), like a receipt, a letter or a screenshot. Tagged `text` and `document`. - **text:** a few words, like a sign or a T-shirt. Tagged `text`. - **none:** no readable words.

Name a folder (a month, a year, or a folder in intake); it reads about 2 to 4 pictures a second:

```
da tag ocr photos\organized\2021                    # a preview: what it finds, nothing changes
da tag ocr photos\organized\2021 --tag-only         # tag them
da tag ocr photos\organized\2021 --tag-and-move     # tag them, and move the documents
```

- **--tag-and-move** asks first, then moves each *document* picture, with its sidecar, to `documents\organized\<year>\<year-month>` with a document's name. One in intake goes to the documents intake folder (`intake\docs`), for **1 Autorun** to file. A picture with no date stays where it is (tagged).
- Every run writes `ocr-results.csv` in `output\runs\<date-time>-ocr\` : each picture, what it held, how many words, what was done, and the start of its text. It opens in Excel.
- The text is kept in the picture's sidecar (`ocr` block). A picture already read is skipped next time (`--again` reads it again), and what a preview read is kept, so `--tag-only` afterwards is quick.
- **Undo the moves:** DA prints the line, `da intake undo "<…\ocr-moved.csv>"` .
- Find them: `da findit tags photos\organized -r document -v` .

Bring in Picasa's faces and captions

If you used Google's Picasa, every folder it looked at has a `Picasa.ini` (or a hidden `.picasa.ini`): the faces it found in each photo and who they are, captions, stars and albums. DA's filing renames the photos and leaves those files behind in intake. `da picasa` copies what they hold into each photo's sidecar:

```
da picasa              # shows what it found; nothing is written
da picasa --apply      # writes it into the sidecars, and keeps the Picasa.ini files in picasa\
```

- **Each photo is found** where it is now: still in intake, filed (DA follows its moves and renames), or - when it was set aside as a copy - the copy DA kept.
- **What it writes:** a `picasa` block in the sidecar: each face's box (where it is in the photo) and name, the caption, the star, the albums. Picasa's email addresses for people are left out.
- **Tags:** `person:jane-doe` for each named face, `album:<name>` , `picasa:starred` . `da tag` keeps them. Find them: `da findit tags photos\organized -r person:jane-doe -v` .
- **The Picasa.ini files** move to `picasa\<the folder they were in>\` in the archive, as a record. Running `da picasa` again still reads them there.
- `picasa-plan.csv` in `output\runs\<date-time>-picasa\` lists every entry: the photo it went to, the people, or why not (*not found*: the photo was deleted, or never copied into this archive). Entries with only Picasa's own edit settings are skipped.
- **Undo:** DA prints the line, `da intake undo "<…\picasa-moved.csv>"` , to put the Picasa.ini files back. The `picasa` blocks stay (running it again rewrites them).
- Faces named in Picasa are the start of face recognition in DA: see da facial.

da facial: find the people in photos

DA can find the faces in your photos and learn who is who from the faces Picasa already named (see above). It suggests names for the other faces; **nothing is named until you confirm it**. Everything happens on this computer: two small face models (from OpenCV, about 40 MB) are downloaded once, and nothing about anyone's face leaves the PC. When a "safe browsing" web filter (in the router, or from the internet provider) blocks OpenCV's download site, DA gets the same files from its own copy on GitHub. If both are refused, DA says why and names the file:

download it in a web browser and save it where DA says (`%LOCALAPPDATA%\DA\models`), then run `da facial scan` again.

```
da facial scan              # find the faces in photos not looked at yet (about a quarter of a se
da facial people            # who's there: photos, faces Picasa named, faces suggested for each p
da facial suggest           # suggest names for the other faces, and a PDF of them to look at
da facial rename "Sam Smyth" "Sam Smith"            # one person under two names: make it one
da facial forget            # delete all face data (or --person "Jane Doe")
```

- **scan** looks at each filed photo once (it remembers which it has seen, by fingerprint, so renames and moves don't count as new). A face Picasa named becomes an example of that person.
- **suggest** compares every other face with each person's examples: *probably Jane Doe*, *maybe Jane Doe*, or not like anyone known - those are put in groups of faces that look alike, to be named once for the whole group. The PDF (`output\runs\<date-time>-facial\faces-<date>.pdf`) shows each suggested face and group, cut from its photo, and `suggestions.csv` lists them. Children's faces look alike more often, so their groups need more checking.
- **Confirming:** open the dashboard (menu **B**) and press **Show** on **People**:
  - **Everyone:** each person and how many photos they're in. Click a name to rename them (two names for one person become one).
  - **Waiting for you:** the faces suggested for each person. Click any that aren't them (they turn red), then **Confirm the ticked ones**: those are named, the unticked ones are never suggested for that person again. **None of them are ...** turns them all down.
  - **Who is this?** groups of faces that look alike. Untick any that don't belong, type the name (names already known are offered) and press **Name the ticked ones**; **Not a face** for things that aren't faces; **Skip**.
  - A name you give is written into the photo's sidecar (`faces` block) and becomes a `person:` tag (`da tag` keeps it). The contact sheets of those photos' folders are remade straight away, so the names show there (and in `da share`), and `da history <photo>` lists who's in it. Run `da facial suggest` again afterwards: each name you confirm makes the next suggestions better.
- Only names you confirmed and Picasa's become `person:` tags.
- A Picasa contact that was saved without a name shows as an account id (`jane.doe_lh,4d61...`); `da facial people` points it out, and `da facial rename` gives it a name.
- The faces found and their numbers are kept in the archive's index only (`.da\da.sqlite`), never in sidecars or shares. If Windows blocks the face part (Smart App Control), `da facial` says so; the rest of DA isn't affected.

da family: where you lived and how old everyone was

Keep the family's homes and birthdays in one folder that all your archives share, e.g. `D:\family` , and name it in each archive's `da-archive.yaml` :

```
family: D:\family
```

It holds two small spreadsheets saved as CSV (Excel: *Save As* > *CSV*), and an optional third:

- **houses.csv** - every place the household lived. Leave **to** blank for the home you live in now; other columns (like a `#yrs` you worked out) are ignored. Several homes with the very same dates are fine (a year of moving between cities). The optional **people** column says whose home it was (`sam` or `jane; sam`, the names in people.csv); leave it blank for the household's homes. Each archive shows its owner's homes (the owner is `archive: owner:`, else the name before `-archive`, so `sam-archive-201x` is sam's): the household's homes, and from the day a home names sam, that one instead. So only a grown child's own homes need a name: `Date,to,address,city,state,zip,people 6/1/1990,8/1/1996,12 Elm Street,Springfield,UT,84000, 8/1/1996,,40 Oak Lane,Riverton,UT,84065, 9/1/2013,,7 Pine Court,Ogden,UT,84401,sam` `da family --person sam` shows anyone's timeline (`--person all`: every home).
- **people.csv** - everyone whose age should show, children, grandchildren and grandparents alike. **also_known_as** lists the other names that person has in your photos (the names on faces from Picasa or `da facial`), separated by `;`, so `Jane Doe` in a photo is matched to `jane`. Capitals and extra spaces don't matter. `name,birthday,also_known_as jane,4/2/1962,Jane Doe; Jane D sam,7/9/1991,Sam Doe; Sammy`
- **members.csv** (optional) - when each person lived with the household. Without it, everyone born before a move-out counts as living in that home; with it, only the people and stretches it lists: `name,from,to jane,, sam,,8/15/2009`

Dates are written 6/1/1990, 6/1/90 (a two-digit year in the future means the 1900s) or 1990-06-01; moving day counts as the new home's.

```
da family                    # check the files and list each home: who lived there and their ages
da family --on 7/4/1998      # the home that day and everyone's age
da family --csv timeline.csv # also save the list as a spreadsheet
da family --folder D:\family # a family folder not named in da-archive.yaml (works outside an arc
```

`da family` points out what looks wrong: a date it can't read, two homes whose dates overlap, one name given to two people, someone with no birthday, and the gaps between one move-out and the next move-in.

Homes and ages under your photos. Once `family:` is set, `da tag` (and Process intake's tags step) works out for every dated photo the home you lived in then and how old each named person in it was, and keeps it in the photo's sidecar. To do it for photos and videos without tagging again (after you change the family files, for example):

```
da family --update                # every filed photo and video (or name folders: da family 201
da family --locate                # once: look up where each home is, so photos with GPS say "a
da contact-sheets                 # remake the sheets to show it
```

- On **contact sheets** a face's label shows the age, `Jane Doe (35)`, and the tile's note line the people and the home: `Jane Doe 35, Sam 6 | 40 Oak Lane (at home)`. `da history <photo>` has a *Family* line, and the dashboard's photo viewer shows it beside the date.

- A date set by hand to just a year or a month (`da date set`) shows ages as a range, `Sam 4-5`, and the home only when you lived in one place all that time.
- A name in a photo is matched to people.csv by its name or its **also_known_as** names; a face whose name isn't there shows no age. Someone in a photo taken before their birthday is skipped (worth a look: the name may be wrong).
- `da family --locate` asks OpenStreetMap for each address once (also tidied: `So` as `South`, an apartment number left out) and keeps the answers in `house-positions.csv` in the family folder, one row per home with its **source**: *found*, *town* (only the town was found - not used, since the middle of town isn't your house), *missing*, or *you*. For a town or missing row, type the house's latitude and longitude into it (in Google Maps, right-click the house); DA marks it *you* and never changes it. `--locate --again` asks again for the town and missing ones; if OpenStreetMap is busy, DA waits and tries again, and keeps every home found so far. A photo within about 300 m of that day's home counts as *at home*; homes with no position of their own don't say either way.
- Homes and ages stay private: `da share` leaves them out unless you add `--with-family`.

Switch between archives

- **6 Switch archive** in the menu lists every archive on this computer. Type its number.
- Or double-click that archive's `Open DA.cmd`.
- Or in a terminal: `da archives` lists them, and `da archives --open 2` opens number 2.

How DA knows your archives: each computer keeps a list in `%LOCALAPPDATA%\DA\archives.json` (on computers that ran DA's older versions, `%LOCALAPPDATA%\MindIt\archives.json`). `da setup` adds a new archive to it, and so does opening an archive (its `Open DA.cmd`, or a command run inside it), so an archive on a USB drive joins the list the first time you open it on that computer. An archive whose folder is gone (a drive that isn't plugged in) is still listed, marked *not connected*, but can't be opened until it's back. (Older versions kept the list in `archives.txt`, one folder per line; DA makes `archives.json` from it the first time.)

For each archive the list keeps its name, folder, the computer it was made on, when it was made and last opened, its years, its owner, its size (GiB), and the places and people its photos show. DA reads those from the archive's index whenever a command changed the index; `da archives --refresh` reads them all again, and `da archives --info 2` shows everything about number 2. The owner is `owner:` in `da-archive.yaml`, else the name before `-archive` (`jane-archive-199x` is jane's).

Update DA

Type `da update`. DA looks for a newer version, says which one, asks, and puts it in place: for DA.exe it downloads the new one, checks its size and fingerprint against the release and that it starts, then swaps it in (DA installed with Python is updated with pip). That's all.

- `da update --check` only says whether a newer DA is out. The menu says so too when one is (it looks at most once a day).

- Windows already open, and a running dashboard, keep the old DA until they're closed. For the dashboard: `da dashboard --stop`, then `da dashboard`.
- If anything goes wrong (no internet, a download that doesn't match), DA says so and nothing changes.
- By hand, if you prefer: download the newest DA.exe and install it the same way as the first time (Getting started).

Your archives aren't touched either way. To see which version you have: `da version`.

Less common workflows

Workflow	What it's for
Videos	Home movies and phone videos, filed beside that month's photos
iPhone Live Photos (.MOV files)	The short videos that come with iPhone photos
iPhone HEIC photos	The iPhone's photo format
Edit an archive's settings	Change its name, years or intake folders
Share an album with Google Photos	Let family see some photos online
Process a folder from somewhere else	A USB stick, an old backup, another drive
Start a new archive for another person or decade	One archive per person per decade
Use DA on another computer	Take an archive on a USB drive to another PC
Merge two copies of an archive	Two drives that each got different photos: make one whole
Back up an archive	Keep a second copy safe
Check what's already filed	A check-up of `organized`
Update sidecars	Bring older `.meta.json` files up to date
Do the run's steps one at a time	For a careful look between each step
Location and GPS workflows	Where photos were taken; folders by place (menu **L**)
Coming soon	What DA will do next

Videos

Put videos in intake with your photos (`intake\photos\...`). **1 Autorun** files them the same way: by when they were recorded, into that month's folder, next to the photos from the same days:

```
photos\organized\2021\2021-06\photo_20210601_090000_ab12.jpg
photos\organized\2021\2021-06\video_20210601_100000_cd34.mov
```

- **Which videos:** `.mov`, `.mp4`, `.m4v`, `.3gp`, `.avi`, `.mpg`, `.mpeg`, `.mts`, `.m2ts`, `.wmv`, `.mkv`, `.webm`.
- **The date** comes from the video file itself: phones and most cameras record it inside. DA reads it without any extra programs. A date in the file name (`VID 00000-20091016-2017.3gp`) is used too.
- **Videos with no date inside** (many old `.mpg`, `.wmv` and some `.avi`) wait in intake under **3 What needs review**, like photos with no date. Give them one with **D Fix dates** (or `da date set <video or folder> --date 2006-06`), then process intake again.
- **Contact sheets** show each video as a dark **VIDEO** tile with its length.
- **The short video of a Live Photo** isn't filed on its own: it goes with its photo (next section).
- `da report` counts videos, `da split` and `da merge` move them like photos, and duplicates are set aside the same way.

iPhone Live Photos (.MOV files)

An iPhone *Live Photo* is two files with the same name: the photo (`IMG_1234.HEIC` or `.JPG`) and a 2-3 second video (`IMG_1234.MOV`).

- **DA keeps the pair together.** When it files the photo, the video goes with it, renamed to match: `photo_20210601_100000_ab12.heic` and `photo_20210601_100000_ab12.mov`, side by side.
- **Videos left behind by an earlier run** (filed before DA did this, or copied in after their photo was filed) can be found and put beside their photo. In a terminal, in the archive: `powershell da pair-movies # shows what it found; nothing moves da pair-movies --apply # moves them` DA matches each video with the photo that was filed from the same place under the same name. It lists what it moved, and `da intake undo "<the file it names>"` puts them back. A video whose photo hasn't been filed yet is left alone; file the photo first.
- **Other videos** (not part of a Live Photo) stay in intake: DA doesn't file videos yet.
- **Voice notes.** Some cameras record a short sound with a photo (`SAM_0010.WAV` with `SAM_0010.JPG`). It follows its photo the same way (`photo_..._ab12.wav`), and `da pair-movies` finds the ones an earlier run left behind too.
- **Don't want Live Photos?** On the iPhone, turn off **Live** (the round icon at the top of the camera) before taking photos.

iPhone HEIC photos

iPhones save photos as **HEIC** (`IMG_1234.HEIC`): the same quality as JPG in about half the space.

- **DA files HEIC photos like any other,** using the date the iPhone stored in them.
- **To see them on Windows,** install the free **HEIF Image Extensions** from the Microsoft Store (open the **Microsoft Store** app and search for *HEIF*).
- **To make JPG copies** (for sharing, or for a program that can't open HEIC): `powershell da heic-to-jpg photos\organized\2021\2021-06` The copies go in a new folder next to it (`2021-06-jpg`), with the same

date taken; add `--out D:\to-share` to put them somewhere else. The HEIC originals stay where they are. Running it again only converts photos that are new or changed.

- **To get JPGs from the iPhone instead,** on the iPhone open **Settings > Photos** and, under *Transfer to Mac or PC*, choose **Automatic**. The iPhone then converts photos to JPG as they're copied. (**Keep Originals** copies the HEIC files.)
- **To find the HEIC files in the archive:** `da db query "SELECT path FROM files WHERE extension='.heic'"`.

Edit an archive's settings (da-archive.yaml)

Each archive's settings are in `da-archive.yaml` in its folder.

1. **Make a copy first:** right-click `da-archive.yaml` > **Copy**, then **Paste** in the same folder. If anything goes wrong, rename the copy back.
2. **Open it with Notepad** (right-click > **Open with** > **Notepad**).
3. **Change what you need.** Keep the spaces at the start of each line exactly as they are; they matter. Lines starting with `#` are notes and are ignored. ```yaml archive: name: "jane-archive-199x" # the name shown at the top of the menu years: "1990-1999" # the years this archive takes; null for any year

   intake: # the folders in intake\ and what each one holds photos: photo docs: document scans: document receipts: document # an extra folder: anything in it is filed as a document

   cameras: # who owned each camera, for owner: tags (see da tag) "Apple iPhone 12 Pro": jane ``` 4. **Save, then check:** open the archive's menu. If DA can't read the file, it says which line is wrong; fix it or put the copy back.

A new intake folder you add here is created the next time you run `da setup` on the archive (`da setup D:\jane-archive-199x`), or you can create it yourself.

Share an album with Google Photos

DA doesn't upload to Google (Google no longer lets apps like DA create shared albums), but your filed photos are easy to share by hand:

1. Go to photos.google.com and sign in.
2. Click **Upload** (or drag photos onto the page) and pick them from the archive, e.g. `D:\jane-archive-199x\photos\organized\1998\1998-07\`.
3. Click **Create** (the **+**) > **Album**, give it a name, and add the uploaded photos.
4. In the album, click **Share** and choose people, or **Create link** to send the link to anyone.

The originals stay in your archive; Google gets copies.

Process a folder from somewhere else

For photos on a USB stick, an old backup drive, or anywhere outside the archive:

```
da run E:\old-backup\photos --archive D:\jane-archive-199x
```

DA offers to **copy** the folder into the archive's `intake\` and processes the copy, so it can check every photo against what's already filed. The originals aren't touched; delete them yourself once you're happy. For a whole drive, or several, da import is quicker: it copies only what no archive has yet.

Start a new archive for another person or decade

```
da setup D:\sam-archive-201x --years 2010-2019
```

Then double-click its `Open DA.cmd`. To make an archive from a folder that already has photos in it, point `da setup` at that folder: DA moves its contents into `intake\`, and lists every move in `output\runs\<date>-setup\moved.csv` (where each item was, and where it went).

Use DA on another computer

An archive is just a folder, so it can live on a USB drive and move between computers.

1. Install DA on the other computer.
2. Plug in the drive and double-click the archive's `Open DA.cmd`. It works whatever drive letter Windows gives the drive.
3. Each computer keeps its own list of archives for **6 Switch archive**. Opening an archive once with its `Open DA.cmd` adds it to that computer's list.

Merge two copies of an archive

Two copies of the same archive on two drives (say `D:\jane-archive-201x` and `E:\jane-archive-201x`) can drift apart: photos added to one, not the other. `da merge` makes one of them whole.

1. **Open the one to keep** (its `Open DA.cmd`), the one that should end up with everything.
2. **See what it would copy:** `da merge --from E:\jane-archive-201x`. DA indexes this archive, compares the other one file by file (by content, not by name), and says: `48,210 files in jane-archive-201x: 47,903 already here. Copy 307 files (2.1 GB) into jane-archive-201x: 291 to the same folder and name here 16 to intake\...\from-jane-archive-201x (their intake; da run files them)`
3. **Press Enter** (or add `--yes`) to copy them. Each file comes with its sidecar.
4. If some went to intake, choose **1 Autorun** to file them.

Good to know: - **The other archive is never changed.** DA only reads it. When you're happy with the merge, you can keep it as a backup or delete it. - **Duplicates aren't copied.** A photo this archive already has, under any name or in any folder, is skipped. - **Where copies go:** a filed photo goes to the same `YYYY\YYYY-MM\` folder and name it had there (location folders too). It goes to `intake\photos\from-<other archive>\` instead, for `da run` to file, when it's from a year outside this archive's years, when a different photo already has that name here, or when the two archives name files differently. - **Changed your mind?** DA prints an **Undo** line: `da intake undo "<the file it names>"` removes the copies again (only copies that haven't changed since). - DA checks there's room on the drive first.

Back up an archive

Everything about an archive is inside its folder: the photos, the documents, the settings, the sidecar notes and the records. DA copies every archive on this computer to a backup drive for you.

Once: plug in the backup drive (say it's `E:`) and type:

```
da backup --setup E:\DA-backup
```

DA marks that folder as this computer's backup and puts a **Back up archives** shortcut on the desktop.

Every week (or after a big day of adding photos): plug in the drive and double-click **Back up archives** (or choose **K Back up** in the menu). The first backup copies everything; later ones copy only what changed, and the copy of each archive ends up the same as the archive. When it says **You can unplug the backup drive**, you're done.

Good to know: - Each archive goes in its own folder: `E:\DA-backup\jane-archive-199x\`. - DA won't back up to a drive that isn't the one you set up (another USB drive can get the same letter), and it stops if an archive suddenly has far fewer files than last time, so a mistake isn't copied over the backup. - To get files back, copy them from the backup folder with File Explorer. - Keep the backup drive somewhere other than the computer when it isn't plugged in.

Move an archive to another drive

Everything about an archive is inside its folder, so moving it is moving that folder, and DA does the rest. Close every DA window (and File Explorer window) in it, open a terminal **outside** it, then:

```
da archives move D:\jane-archive-199x E:\jane-archive-199x
```

- **On the same drive** it's one rename: instant.
- **To another drive** DA copies everything (photos, documents, sidecars, its index, logs and records), then checks every file arrived: the same size and date for all, and the full contents for a sample (`--full` re-reads every file; slow on a big archive). If anything is wrong, it stops, and the archive is still where it was.

- DA's records follow it: the index, the undo logs (so `da intake undo` still works), and this computer's list of archives. `Open DA.cmd` works from the new place as it is.
- **The old copy** (another drive): DA asks whether to send it to the Recycle Bin. If you keep it, it's no longer an archive (a `MOVED.txt` in it says where it went), so nothing gets filed into it by mistake; delete it in File Explorer when you're sure. `--remove-old` sends it to the Recycle Bin without asking.
- The new place must be a new folder, or an empty one, with room for the archive.
- On another computer that also uses the archive (a USB drive), open it once from its new place (its `Open DA.cmd`) and it's on that computer's list too. That computer can only look at it until you make it the archive's home.

One computer changes an archive (its home)

Each archive has a **home**: the one computer that files, renames, tags and indexes it. `da setup` makes the computer you set it up on its home (the `home:` line in `da-archive.yaml`). Why: if two copies of an archive (say one on your desktop and one on your laptop) were both changed, DA could not put them back together.

On any other computer, DA only **looks**: the dashboard, `da history`, `da find`, `da share`, `da report` and `da status` work, and the menu shows `home: DESKTOP (this PC only looks)`. Anything that would change the archive (`da run`, `da tag`, `da rename`, `da location`, menu choices like **1 Autorun**, naming people on the dashboard) stops and says which computer to use instead.

See or change an archive's home, in the archive's folder:

```
da archives home              # shows its home and this computer's name
da archives home LAPTOP       # LAPTOP changes it from now on (asks first)
```

- Change the home only when the archive itself goes with you (a USB drive, or you moved it with `da archives move`). After that, **don't change any other copy** of it.
- New photos from a trip, on a computer that isn't the home: put them in a folder outside the archive, then copy them into the archive's `intake\` on its home computer and process them there.
- `da archives` lists each archive's home. An archive made before this has none, and any computer can change it until you set one. Until then, each command that changes it starts with *This archive has no home PC* (and the menu shows `home: not set`); `da archives home <this computer's name>` sets it.

Delete an archive you no longer need

After its photos have gone to other archives (with da split --organized or merge), delete the old archive with DA, which makes sure nothing is lost first. Open a terminal **outside** that archive (or another archive's `Open DA.cmd`), then:

```
da archives delete D:\jane-archive-all
```

1. DA checks **every** photo, video, document and raw file in it (organized, intake and `_duplicates-holding`) by its fingerprint, against the other archives on this computer. A backup drive counts too: `--also F:\backup`. It writes `delete-check-<name>-<date-time>.csv`: each file and where its copy is.
2. **If any file exists only there,** it stops, lists the first few, and deletes nothing. Move them first (`da split --organized`, or `da merge --from` in the archive to keep), then run it again.
3. **If everything is elsewhere,** it asks you to type the archive's name, then sends the whole folder to the **Recycle Bin**, and takes it off this computer's list. If the folder is too big for the Recycle Bin, Windows asks before deleting it for good: answer **No** to keep it.

- Only on a computer's own drives: a USB or network drive has no Recycle Bin, so DA only checks there; delete it in File Explorer yourself.
- `da archives delete D:\jane-archive-all --forget` only takes it off this computer's list; its files stay.

da history: what DA did, and when

DA writes down everything it changes: every file it files, renames, moves or sets aside (the move logs in `output\runs\`), and every command's summary, with the computer it ran on (`logs\da.jsonl`). `da history` puts those together, newest first:

```
da history                          # the latest changes in this archive
da history --kind video             # only videos (or photo, document, raw)
da history --since 2026-10-01       # only changes from that day on
da history --limit 200              # more of them (50 by default)
da history --no-save                # just show it
```

- **First** each command's summary: *2026-10-18 09:41 run of intake: filed 38,386 ... (FOXTROT)*.
- **Then** the files: what happened (*filed*, *renamed*, *set aside (a copy in intake)*, *moved to another archive*, *moved into a location folder* ...), by which command, from where to where. A change you undid (`da intake undo`) is marked *undone*.
- **Saved** as `history.csv` (opens in Excel) and `history.json` in `output\runs\<date-time>-history\`.
- The first time, DA reads all its records (a few seconds on a big archive); after that only what's new.
- History starts from DA's own records (your first `da run`). Where a photo was before DA, its original name and folder, is kept in its sidecar.

One file's whole story. Name the file - its path, its name now, or the name it had before DA renamed it - or the first 8 or more characters of its fingerprint (SHA-256, shown by `da history` and in its sidecar):

```
da history IMG_1234.HEIC                                    # by the name it came in with
da history photos\organized\2021\2021-06\photo_20210601_100000_142f.jpg
da history --hash 142fdc1d                                  # by its fingerprint
da history IMG_1234.HEIC --everywhere                       # and its copies in every archive on th
```

- *Now:* where it is, its kind and size, when it was taken (and *set by hand* if you did), camera and owner, place, fingerprint.
- *Came from:* the name and folder it had before DA.
- *History:* every change, oldest first, followed through moves and renames: *filed (da run)*, *renamed (da rename)*, *moved into a location folder* ... with undone ones marked.
- *Copies:* the same photo elsewhere in this archive (organized, intake, `_duplicates-holding`), and with `--everywhere` in your other archives too (an archive DA hasn't indexed yet has nothing to show).
- A file DA moved away (to another archive, or set aside and then removed) shows *Not in the index now* and where it was last.

Don't know the file's name? Find files by the folder they came from, the camera that took them, or part of a name. Part of each is enough, upper or lower case doesn't matter, and you can use them together:

```
da history --from 100CANON                       # everything that came from a camera's DCIM\100C
da history --from "phone 2024"                   # a folder you copied photos into (phone-2024-su
da history --camera powershot                    # everything a Canon PowerShot took
da history --camera "iphone 12" --from 101APPLE  # both at once
da history --name IMG_00                         # IMG_0001.JPG ... IMG_0099.JPG, as they came in
da history --from 100CANON --everywhere          # in every archive on this PC
```

- DA shows how many it found, then how many by **camera** and by **folder they came from**, then the files (oldest first) with where each one is now. One file found: its whole story.
- `--limit 200` shows more (50 by default). All of them are saved in `history.csv`.
- A space, `-` or `_` matches any of them (`phone 2024` finds `phone-2024`).
- Camera names: `da tag --cameras` lists the cameras in this archive.
- **Cameras the first time:** the camera is kept in the index from this version on. If DA says the index doesn't know cameras yet, run **5 Refresh index** (`da db build`) once. Processing intake does it too.

Every held copy. `da history --duplicates` lists each file in `_duplicates-holding`, when it was set aside, and the file it's a copy of - handy before you empty that folder. Add `--everywhere` for all your archives.

Check what's already filed

```
da validate photos\organized
da validate documents\organized\2010
```

This checks names, folders and sidecar notes, and finds duplicates inside that folder. It's a check only: nothing moves (it just adds any missing sidecar notes, and remakes any contact sheet whose folder has changed). The report is saved in `output\validateit\`.

To fix what it found in `organized` (wrong names, a photo in the wrong month), add `--mode repair`. On intake, or on a folder outside the archive, repair stops and tells you to use da run instead, which sets duplicates aside first.

Update sidecars (the .meta.json files)

Beside every file DA keeps a small **sidecar**, `photo_20210601_100000_ab12.jpg.meta.json`, with what it knows about the file: when it was taken, its fingerprint, its tags, its place. Sidecars made by older versions of DA (or its predecessor) are laid out the old way, with leftover fields. They still work, but they're cluttered.

1. Choose **J Update sidecars** (or type `da sidecar upgrade`). DA reads every sidecar in the archive and says how many are out of date, and what would change: `143,021 sidecars: 35,810 up to date (schema 2), 107,211 older. 107,211 schema 1 -> 2 107,211 empty scanit block left out 80,412 old fields at the top moved into their blocks`
2. Press **Enter** to update them. Nothing is lost: old values that differ from the current ones are kept under `legacy`. Your tags, dates, places and titles stay as they are, and the photos themselves aren't touched.
3. A copy of every sidecar it changed is kept in `output\runs\<date-time>-sidecars\backup\`. To put them back: `da sidecar upgrade --restore "<that folder>"` (DA prints the exact line).

You don't have to do this: any sidecar DA changes for another reason (validating, tagging, a date or a place) is updated at the same time. To do just one folder: `da sidecar upgrade photos\organized\2019`.

Do the run's steps one at a time

The same steps as da run, one command each, for when you want to look closely between them:

```
da db build                                          # 1 index
da intake hash --folder phone-2024                   # 2 hash (folder inside intake)
da intake set-aside --folder phone-2024              # 3 plan only: what would be set aside
da check output\runs\<ts>-set-aside\plan.csv         #   re-check a sample of the plan
da intake set-aside --folder phone-2024 --apply      #   set them aside
da validate intake\phone-2024 --kind photo           # 4 preview
da check output\validateit\validateit-report-<ts>.json   # 5 check
da validate intake\phone-2024 --kind photo --mode repair --force   # 6 file (--force: you did 2-3
da verify <the repair report it printed>             #   quick: place, size, date, a sample re-ha
da verify <the repair report> --full                 #   every file re-read (slow: about 1 hour p
da db build                                          # 7 refresh
```

Move folders to another archive by hand

When a few months (or years) belong in another archive and you'd rather move them yourself. Each photo's `.meta.json` sidecar is in the same folder as the photo, so moving whole folders keeps them together.

1. **Close DA** in both archives (every `Open DA.cmd` window), and stop the dashboard: `da dashboard --stop`.

2. **Copy the folder** to the same place in the other archive, e.g. from `D:\jane-archive-201x\photos\organized\2019\2019-04` to `E:\sam-archive-201x\photos\organized\2019\2019-04`. In File Explorer, copy and paste; or in PowerShell, which keeps the files' dates: `powershell robocopy "D:\jane-archive-201x\photos\organized\2019\2019-04" "E:\sam-archive-201x\photos\organized\2019\2019-04" /E /COPY:DAT /DCOPY:DAT`
    - If the two archives name or folder files differently (see **N Naming** in each), copy into the other archive's `intake\photos\` instead, and file them there with **1 Autorun**.
    - A location folder (`2019-04 Easter (springfield-il)`) moves the same way, next to its month.
3. **Check the copy**, in the other archive: `powershell da validate photos\organized\2019\2019-04` It should find nothing to fix. In File Explorer, **Properties** of the old and new folder show the same number of files and size.
4. **Delete the original** folder in File Explorer (it goes to the Recycle Bin; keep it there until you've checked everything).
5. **Refresh both indexes:** choose **5 Refresh index** in each archive (or `da db build`).
6. **Contact sheets:** in the other archive, `da contact-sheets photos\organized\2019\2019-04`. In the old archive, delete the year folder if it's now empty.
7. **Check again:** `da rename` in the other archive says *0 moved to another folder*.

- Moves you make by hand have **no undo log**: the Recycle Bin is your way back until you empty it.
- Copies of those photos set aside in `_duplicates-holding` stay in the old archive.
- `da split --organized` (see da split) does all of this for every file another archive's years take.

Coming soon

- **Dates for scanned prints:** a suggested date for an undated scan, from the people in it and their birthdays (da family holds the homes and birthdays).

Location and GPS workflows

Most phones record **where** each photo was taken (its GPS position); older cameras usually don't. DA reads the position when it first files a photo (or when you run da tag on photos filed earlier) and keeps it in the photo's sidecar.

Works today

- **See which photos have a position:** `da tag` (reads it from each photo once), then `da findit tags photos\organized -r gps -v`.
- **Find the ones without:** `da findit tags photos\organized -x gps -v`.
- **Put the position in file names:** use `{gps_compact}` (`N39W89`) or `{gps_lat_dir:2}{gps_lon_dir:2}` (`N39.78W89.65`) in the name pattern; see Config tokens.

Look up where photos were taken

`da location lookup` turns each photo's position into a town (or city) and state, and tags the photo with it: `place:springfield-il`, `state:il`, `country:us`.

1. Open a terminal in your archive (double-click `Open DA.cmd`).
2. Type `da location lookup` for all your filed photos, or name a folder: `da location lookup photos\organized\2019`.
3. DA says how many photos have a position and how many new places it has to look up, about one a second (1,000 new places: about 17 minutes). Press Enter to start.
4. When it's done it lists the places it found, with how many photos each. A lookup never moves a photo: it ends with the command that makes location folders for the months it found, like `da location group 2024`.

Good to know: - Photos close together (within about 100 m) share one lookup, and every lookup is remembered in the archive, so running it again later only looks up new photos. Run it after each `da run`. - Photos filed before DA read camera details need `da tag` first; DA tells you if none have a position. - No internet? The photos it couldn't look up just wait; run `da location lookup` again later. `da location lookup --offline` uses only the places already remembered. - Photos taken at sea or in the wild may have no town; DA says how many.

Then find photos by place: - `da findit tags photos\organized -r place:springfield-il -v` (by town), `-r state:il` (by state), `-r country:fr` (by country). - Or ask the index: `da db query "SELECT path FROM files WHERE city = 'Springfield'"`.

Group a month into location folders

Instead of one big `2019-04` folder, put each outing in its own folder, named by you and by where it was: `2019\2019-04 Easter at grandma's (springfield-il)\`.

1. In the menu choose **L Location folders**. DA lists the months that have GPS photos; type a number. (Or type `da location group 2019-04`, or the month's folder, `da location group photos\organized\2019\2019-04`.)
2. DA looks up any places it doesn't know yet, then shows the **groups**: photos taken within 6 hours and about 2 km of the one before belong together, so a day out, or a weekend in one place, is one group. Each line shows how many photos, when, and where: `1 24 photos Sat 20 Apr 10:00 - Sat 20 Apr 16:40 springfield-il (24) 2 9 photos Sat 27 Apr 15:00 chicago-il (9)`
3. Type a **title** for each group you want a folder for, like `Easter at grandma's`. Press Enter to leave a group in the month folder. Give two groups the **same title** to put them in one folder.
4. DA shows the folder names it will make, using your folder pattern, and asks before it moves anything. Press Enter to go ahead.
5. The photos move, with their sidecars and any raw files or Live Photo videos, and each new folder gets its contact sheet. DA prints an **Undo** line: to put everything back, type `da intake undo "<the file it names>"`.

Good to know: - Photos without GPS stay in the month folder, unless you give them a place first, or turn on `without_gps` (below). If the month folder ends up empty, DA removes it. - Groups too big or too small? `--hours 3` or `--km 0.5` makes smaller groups; `--hours 12 --km 10` makes bigger ones: `da location group photos\organized\2019\2019-04 --hours 3`. - `da validate` and `da run` leave photos in a location folder of their own month. - **New photos later?** `da run` files them in the plain month folder. Choose **L** on that month again: a group taken at the same time and place as a folder you already made says `fits the folder already there`, and pressing Enter adds the photos to it (type `-` to leave them). - Without the menu, give the titles in the command: `da location group photos\organized\2019\2019-04 --title "1=Easter at grandma's" --title "2=Zoo"`. A title that is an existing folder's title adds the photos to that folder.

A name for groups you don't title. Add `untitled` to the `location:` section of `da-archive.yaml`, and a group you press Enter for gets a folder named that way instead of staying in the month folder:

```
location:
  folder: '{YYYY}-{MM} {title} ({place})'
  untitled: '{city}'                # or '{camera}-{owner}', '{city} {state:2}' ...
```

`untitled`	**A group from Springfield, Illinois, taken with Jane's Canon, goes in**
(not set)	the month folder, `2026\2026-03\`
`'{city}'`	`2026\2026-03 Springfield (springfield-il)\`
`'{camera}-{owner}'`	`2026\2026-03 Canon PowerShot A80-jane (springfield-il)\`
`'{city} {state:2}'`	`2026\2026-03 Springfield IL (springfield-il)\`

- DA shows the name as each group's answer: press **Enter** to take it, type your own title instead, or type **-** to leave that group in the month folder.
- **Each place keeps its own folder.** Groups that get the same name this way (every group is `Canon PowerShot A80-jane`) only share a folder when the whole folder name is the same - the same places too - so a month in three towns makes three folders. A title you **type** twice still joins those groups.
- The values are written the way they're named, like a title you'd type: the city most of the group's photos were taken in, and the camera and owner most of them came from (`da tag --cameras` sets owners). Tokens: `{city}` `{county}` `{state}` `{state:2}` `{country}` `{country:2}` `{place}` `{camera}` `{owner}` `{event}` `{YYYY}` `{MM}` `{DD}`. One with nothing (a camera that didn't record its name) is left out with its `-`; a group with nothing at all stays in the month folder.
- Without the menu, `da location group 2026-03 --yes` names every group that way (with `--title` for some, the rest get the `untitled` name). Without `--yes` it only shows what it would do.

Photos without GPS, and the event's name. Scans, older cameras and phones with location turned off don't record where a photo was taken. With `without_gps: true`, DA groups those too, by time alone, and `{event}` offers the name of the folder the photos came in from, without its date:

```
location:
  untitled: '{event}'
  without_gps: true
```

- Photos that came in from `intake\photos\2019-04-20 Easter\` are offered **Easter**: press Enter, and they go in `2019\2019-04 Easter\` (with the place in brackets when they have GPS).
- That makes naming events easy: **make a folder named for the event in `intake\photos`, put the photos in it, and choose 1 Autorun** with **11 Location folders** switched on in **P Pre-run setup**. With **Bypass yes** on too, DA asks nothing and the photos go straight into their event folder.
- A photo with only a year or a month known (a date you set with **D Fix dates**) isn't grouped.
- New archives have both settings.

One answer for every group (no question per group; DA still asks before moving, unless `--yes`):

```
da location group 2026-03 --skip-title            # each group gets its untitled name, a folder
da location group 2026-03 --set-title "Spain trip"  # each group titled Spain trip, a folder per
```

- `--set-title "Spain trip"` makes `2026-03 Spain trip (rota-es)`, `2026-03 Spain trip (tarifa-es)` ... one folder per place. To put them all in **one** folder instead, answer each group with the same title.
- `--skip-title` needs `untitled` in `da-archive.yaml`; without it, DA says so and nothing moves.

A whole year at once: give a year instead of a month, and DA does each of its months that still has GPS photos not in a location folder, one after another (menu **L** takes a year too):

```
da location group 2013 --skip-title --yes         # every month of 2013, no questions
da location group photos\organized\2013           # the year's folder works too; asks for each mo
```

`--title "1=..."` numbers one month's groups, so it isn't taken with a year. - A group that fits a folder you already made still goes into it, and `--title "2=Zoo"` still names group 2.

Rename a location folder

Typed the wrong title, or thought of a better one?

```
da location rename "photos\organized\2019\2019-04 Easter (springfield-il)" "Easter at grandma's"
```

DA shows the new name, `2019-04 Easter at grandma's (springfield-il)`, and asks first. The month and places in the name stay; the title changes in the folder name, in every photo's sidecar, in the index and on the contact

sheet. It prints the command that renames it back.

Set a place by hand

For photos with no GPS (an old camera, a scan of a print), or when the looked-up place is wrong:

- **By name:** `da location set "photos\organized\2019\2019-04\photo_20190420_150000_ab12.jpg" --place "Springfield, IL"`. DA looks the name up, shows what it found, and asks first. A town and state, or a town and country (`"Paris, France"`), works best.
- **Same as another photo:** `--same-as <a photo from that day>` copies its place. Nothing is looked up.
- **A whole folder:** name the folder instead of a photo.
- **Take it off again:** `da location set <photos> --clear`. The next `da location lookup` looks the photo up again (if it has GPS).

A place you set is never replaced by a lookup. Photos without GPS that have a place join a group when you group the month, as if taken at that place.

Privacy: looking up places sends each spot's position (rounded to about 100 m, never the photo or its name) to OpenStreetMap's free lookup service, Nominatim. To keep positions on this computer, set `lookup: off` under `location:` in `da-archive.yaml`:

```
location:
  lookup: off
```

Config tokens

A **token** is a word in curly brackets, like `{hash}`, that DA replaces with something about the photo. Tokens make up the patterns DA names files (and, soon, folders) with. The patterns are in the archive's da-archive.yaml:

```
modules:
  photoit:
    pattern: photo_{date_taken:%Y%m%d_%H%M%S}_{hash:4}      # photos
  scanit:
    pattern: doc_{date_taken:%y%m%d_%H%M%S}_{hash:4}        # documents
```

File-name tokens

Token	Example	What it is
`{date_taken:%Y%m%d_%H%M%S}`	`20190420_170509`	when it was taken, written with the date codes below
`{year}` `{month}` `{day}`	`2019` `04` `20`	parts of the date
`{hash}` / `{hash:4}`	`ab12cd34...` / `ab12`	the photo's fingerprint (`:4` = its first 4 characters)
`{orig_name}` / `{orig_stem}` / `{orig_ext}`	`IMG_1234.HEIC` / `IMG_1234` / `heic`	its name before DA renamed it
`{orig_folder}` / `{orig_folder:10}`	`Easter 2019` / `Easter 201`	the folder it came from (`:10` = first 10 characters)
`{from:1}` / `{from:2}` / `{from:-1}` / `{from}`	`Family` / `Easter 2019` / `Easter 2019` / `Family - Easter 2019`	the folders it came in from: the top one, the next, the last, all of them (see Use the folders it came from)
`{tag}` / `{tags}`	`beach` / `beach_grandma`	your first tag / all your tags (da tag)
`{gps_compact}`	`N39W89`	where it was taken, whole degrees
`{gps_lat_dir:2}{gps_lon_dir:2}`	`N39.78W89.65`	the same, to 2 decimal places
`{gps_lat:4}` / `{gps_lon:4}`	`39.7817` / `-89.6501`	the position as plain numbers
`{YYYY}` `{MM}` `{DD}` `{hh}` `{mm}` `{ss}`	`2019` `04` `20` `17` `05` `09`	parts of the date and time
`{maker}` / `{camera}` / `{lens}`	`apple` / `apple-iphone-12-pro` / `iphone-12-pro-back-camera-4.2mm-f-1.6`	the camera that took it (camera details)
`{owner}`	`jane`	who used that camera (`da tag --owner`), else the owner the camera wrote in
`{place}`	`springfield-il`	where it was taken (da location lookup)
`{city}` `{county}` `{street}`	`springfield`	parts of the place
`{state}` / `{state:2}`	`illinois` / `il`	
`{country}` / `{country:2}`	`united-states` / `us`	
`{title}`	`easter_at_grandma's`	its location group's title (location folders)
`{kind}` / `{ext}`	`photo` / `heic`	photo, video or document; the file's extension
`{text}`	`document`	`document` or `text` when da tag ocr found words in it

Date codes for `{date_taken:...}` :

Code	Example		Code
`%Y`	`2019`		`%d`
`%y`	`19`		`%a` / `%A`

Code	Example		Code
`%m`	`04` (month)		`%H` `%M` `%S`
`%b` / `%B`	`Apr` / `April`		`%j`

Examples for a photo taken on 20 April 2019 at 17:05:09, first named `Easter 2019\IMG_1234.HEIC`, tagged `beach` and `grandma`, taken at 39.78 N, 89.65 W:

Pattern	File name
`photo_{date_taken:%Y%m%d_%H%M%S}_{hash:4}` (the default)	`photo_20190420_170509_ab12.heic`
`{date_taken:%Y%m%d}_{hash:8}`	`20190420_ab12cd34.heic`
`{date_taken:%Y-%m-%d_%H%M%S}`	`2019-04-20_170509.heic`
`{date_taken:%Y-%m-%d}_{orig_stem}`	`2019-04-20_img_1234.heic`
`{orig_stem}_{date_taken:%Y%m%d}`	`img_1234_20190420.heic`
`{year}-{month}-{day}_{orig_folder}_{hash:4}`	`2019-04-20_easter_2019_ab12.heic`
`{date_taken:%Y-%m-%d (%a) %H.%M}_{hash:4}`	`2019-04-20_(sat)_17.05_ab12.heic`
`{date_taken:%Y%m%d}_{tag}_{hash:4}`	`20190420_beach_ab12.heic`
`{date_taken:%Y%m%d_%H%M%S}_{tags}`	`20190420_170509_beach_grandma.heic`
`{date_taken:%y%m%d_%H%M}_{gps_compact}`	`190420_1705_N39W89.heic`
`{date_taken:%Y-%m-%d}_{gps_lat_dir:2}{gps_lon_dir:2}_{hash:4}`	`2019-04-20_N39.78W89.65_ab12.heic`
`{date_taken:%Y%m%d}_{gps_lat:4}_{gps_lon:4}`	`20190420_39.7817_-89.6501.heic`
`{YYYY}{MM}{DD}_{camera}_{hash:4}`	`20190420_apple-iphone-12-pro_ab12.heic`
`{YYYY}{MM}{DD}_{owner}_{place}_{hash:4}`	`20190420_jane_springfield-il_ab12.heic`

Rules: - Names are made lower case and spaces become `_` (compass letters like `N39W89` stay capitals). - A token with nothing to show (no tags, no position, no owner) disappears, with its extra `_`. - Keep `{hash}`, or the time to the second, in the pattern: without it two photos can end up with the same name. - **Change a pattern with menu N Naming** (or `da rename`, below): it renames the files already filed to match, and can be undone.

Change how files are named: N Naming, da rename

Choose **N Naming** in the menu. It shows each kind's pattern (photos, videos, documents) and the ones used before. Then: - **Enter** checks every filed file against the patterns, and offers to rename those that don't match. - **A number** (1 photos, 2 videos, 3 documents) asks for a new pattern for that kind, shows what every file would be called, asks *Rename N files?*, then *Save it in da-archive.yaml?*

The same in the terminal:

```
da rename                                                          # check: what would change (
da rename photos\organized\2019 --pattern "{YYYY}{MM}{DD}_{camera}_{hash:4}"           # try a p
da rename --pattern "{YYYY}{MM}{DD}_{camera}_{hash:4}" --kind photo --apply --save     # rename,
```

It prints how many would be renamed (with the raw files and Live Photo videos that follow them), how many are already named so, and what to follow up: - **conflicts:** two files would get the same name; they keep theirs (put `{hash}` in the pattern). - **no sure date / no sidecar:** they keep their names; set a date with `da date set`, or run `da validate` on the folder, then try again. - **had nothing for {owner}** (or another token): it was left out of those names.

- **The plan**, file by file, is `rename-plan.csv` in `output\runs\<date-time>-rename\` (it opens in Excel).
- **--apply** asks, then renames each file with its sidecar, raw file and Live Photo video, updates the index, and remakes the folders' contact sheets.
- **Undo:** DA prints the line, `da intake undo "<...\rename-moved.csv>"`; the old names come back.
- **--save** writes the pattern into `da-archive.yaml` (with `--kind`), and keeps the earlier ones as notes under it, newest first, so going back is easy: `yaml pattern: "{YYYY}{MM}{DD}_{camera}_{hash:4}" # was (2026-10-16): photo_{date_taken:%Y%m%d_%H%M%S}_{hash:4}` New files are named by it from then on. A copy of the old file is kept (`da-archive.yaml.bak-<date-time>`).

Choose the folders too: folder patterns

Where filed files go is a pattern as well, with the same tokens, and `\` between folder levels. The default is the layout DA has always used, `organized\2019\2019-04\`:

```
modules:
  photoit:
    pattern: photo_{date_taken:%Y%m%d_%H%M%S}_{hash:4}
    folder: '{YYYY}\{YYYY}-{MM}'           # the default (videos use the photos' folder unless th
```

Quotes and `\` in `da-archive.yaml`: put a folder pattern in single quotes (`'{YYYY}\{YYYY}-{MM}'`) and write each `\` once. In double quotes, write each `\` twice (`"{YYYY}\\{YYYY}-{MM}"`): it's the same pattern, and it's how DA writes one when it saves it for you (**N Naming**, `da rename --save`). A single `\` in double quotes stops the file being read, and DA says which line. The patterns in the tables below are the patterns themselves, with one `\`.

Folder pattern	Where a photo of 20 April 2019, taken with Jane's iPhone in Springfield, goes
`{YYYY}\{YYYY}-{MM}` (the default)	`organized\2019\2019-04\`
`{owner}\{YYYY}\{YYYY}-{MM}`	`organized\jane\2019\2019-04\`
`{YYYY}\{camera}`	`organized\2019\apple-iphone-12-pro\`
`{country:2}\{state:2}\{YYYY}`	`organized\us\il\2019\`
`{YYYY}\{YYYY}-{MM}-{DD}`	`organized\2019\2019-04-20\`
`{YYYY}\{YYYY}-{MM}-{from:1}` (it came in as `Family\Easter 2019\IMG_1234.HEIC`)	`organized\2019\2019-04-Family\`
`{from}\{YYYY}`	`organized\Family\Easter 2019\2019\` (every folder it came in from)

- **Try one first:** `da rename --folder "{owner}\{YYYY}\{YYYY}-{MM}"` shows where every file would go, and moves nothing. Add `--kind photo --apply --save` to move them and keep the pattern (the old one stays as a `# folder was (...)` note). Or use **N Naming**: after the name pattern it asks for a folder pattern.
- **A token with nothing** (no owner, no place) drops its level: with `{owner}\{YYYY}`, a photo with no known owner goes in `organized\2019\`. `da rename` counts them (*had nothing for {owner}*) and notes each one in the plan, so you can fill them in (`da tag --owner`, `da location lookup`) and run it again.
- **Location folders** work in any layout (`da location group 2019-04` finds the month's photos wherever they are), and stay as they are: a photo grouped into `2019-04 Easter (springfield-il)` goes in that folder in place of the pattern's last level (`jane\2019\2019-04 Easter (springfield-il)\`), or where you put `{location}` in the pattern. A folder named like the expected one plus more (`2019-04 Easter at grandma's` for `2019-04`) is always accepted.
- New files are filed by the pattern from then on, `da validate` checks against it, and folders a move leaves empty are removed. `da intake undo` puts everything back.
- Characters Windows doesn't allow in folder names (`: * ? " < > |`) are left out.

A photographer's edits: beside their RAW

Lightroom and Photoshop exports say which RAW they were made from. Once `da edits --apply` (or processing intake) has paired them, each edit is filed beside its RAW and named after it - every export kept. Choose how in `da-archive.yaml`:

```
modules:
  photoit:
    edits:
      folder: ''                  # '' beside the RAW, or '{Purpose}' for a subfolder per purpos
      name: '{raw}-{purpose}'     # or '{raw}-edit{n}' (numbered, largest first)
```

Setting	The RAW `photo_20210423_162139_f788.cr2` and its exports
`name: '{raw}-{purpose}'` (the default)	`..._f788-print.jpg`, `..._f788-social-media.jpg`, `..._f788-screen.jpg`
`name: '{raw}-edit{n}'`	`..._f788-edit1.jpg`, `..._f788-edit2.jpg`, `..._f788-edit3.jpg`
`folder: '{Purpose}'`, `name: '{raw}-edit'`	`Print\..._f788-edit.jpg`, `Social Media\..._f788-edit.jpg`

`{purpose}` comes from the export's folder (`For Print`, `For Social Media`, `For Screen`, `Web`); when no folder said, an export is numbered (`-edit2`). Two exports that would get the same name get `-2`, `-3`. An edit whose RAW isn't in the archive is filed like any photo. The export's own name stays in its sidecar.

Contact sheets say `RAW + 3 edits (print, social media)` under a RAW and `edit of IMG_1732.CR2 (print)` under an edit; `da share` and `da history` show the same line. When a RAW moves into a location folder, its edits go with it.

A JPG for each RAW: da convert

A camera RAW (`.cr2`, `.cr3`, `.nef`, `.arw`, `.dng` ...) shot without a JPG opens in few programs, and face finding and similar-picture checks don't look at RAWs. Every camera keeps a full-size JPEG of the picture inside the RAW; `da convert` takes it out and puts it beside the RAW with the RAW's name:

```
da convert                          # shows which RAWs would get one (photos and intake)
da convert --apply                  # makes them
da convert "intake\photos\Smith wedding" --apply      # only these folders
```

- `photo_20141116_135202_2cd6.cr2` gets `photo_20141116_135202_2cd6.jpg`: the pair is filed, moved and named together from then on, as if the camera had saved both. The picture is the camera's own JPEG (not developed again), with the RAW's date, camera, lens and GPS.
- Each JPG's sidecar says it was made (a `converted` block, the tag `converted`) and carries the RAW's origin folders, place, people and your tags.
- Skipped: a RAW that has a JPG already, one with a photographer's edits (`da edits`), and one with only a small thumbnail inside.
- Each JPG takes about 1-3 MB, so 10,000 RAWs need 10-30 GB more. `da intake undo` with the log it names deletes the JPGs it made.

Use the folders it came from

The folders you sorted photos into before DA keep their meaning. DA writes them in each photo's sidecar when it first sees it (the `origin` block), every level from the top, and never changes them afterwards: `intake\photos\Jones Family Photos\Scans\IMG_0003.JPG` came from `Jones Family Photos`, then `Scans`.

- **In folder and name patterns:** `{from:1}` is the top folder (`Jones Family Photos`), `{from:2}` the next (`Scans`), `{from:-1}` the last one, and `{from}` all of them - in a folder pattern as folders inside each other,

in a file name joined by `-`. A level that isn't there is left out, like any empty token. `yaml modules: photoit: folder: '{YYYY}\{YYYY}-{MM}-{from:1}' # organized\2009\2009-03-Jones Family Photos\` Try it first: `da rename --folder "{YYYY}\{YYYY}-{MM}-{from:1}"` shows where each photo would go and moves nothing.

- **As tags:** da tag gives each photo a `folder:` tag for every folder it came in from (`folder:jones-family-photos` , `folder:scans`), so you can find them: `da findit tags photos\organized -r folder:jones-family-photos -v` . Processing intake does it too.
- **Left out of tags** (they stay in the sidecar, and `da history --from` still finds them): a camera's own folders (`DCIM` , `100CANON` , `101APPLE`), folders that are only numbers (`2016` , `07` , `2009-01`), months like `06 - Jun` , and everyday ones (`Pictures` , `Photos` , `Documents` , `Camera Roll` , `New folder`). Add your own in `da-archive.yaml` (`*` stands for any text), then run `da tag` again: `yaml origin: skip: ["* working", inbox, backup]`
- **Photos filed before this version:** `da tag` writes their `origin` block from the path kept in their sidecar (a photo with no recorded path gets none), then the tags. **5 Refresh index** fills the index's `origin` column.

Folder tokens

`da run` files photos by the folder pattern (by default `organized\YYYY\YYYY-MM\`). Location folders (see Group a month into location folders) are named by a folder pattern, set in `da-archive.yaml` :

```
location:
  folder: "{YYYY}-{MM} {title} ({place})"      # the default
```

The two folder settings work together. `modules.photoit.folder` is the whole path every photo is filed in; `location.folder` is only the name of a location folder that `da location group` makes. A grouped photo's location folder takes the place of the last level of the photo pattern:

`modules.photoit.folder`	**A photo grouped as** `2019-04 Easter at grandma's (springfield-il)` **goes in**
`'{YYYY}\{YYYY}-{MM}'` (the default)	`organized\2019\2019-04 Easter at grandma's (springfield-il)\`
`'{YYYY}\{camera}\{YYYY}-{MM}'`	`organized\2019\apple-iphone-12-pro\2019-04 Easter at grandma's (springfield-il)\`
`'{YYYY}\{location}\{camera}'`	`organized\2019\2019-04 Easter at grandma's (springfield-il)\apple-iphone-12-pro\` (`{location}` puts it there instead)

Photos that aren't in a group just follow `modules.photoit.folder` .

Token	Example	What it is
`{YYYY}` `{MM}` `{DD}`	`2019` `04` `20`	the date of the group's first photo
`{title}`	`Easter at grandma's`	what you type for the group (kept as typed)
`{street}`	`maple-street`	the road
`{neighbourhood}`	`old-town`	
`{city}`	`springfield`	city, town or village
`{county}`	`sangamon-county`	
`{state}` / `{state:2}`	`illinois` / `il`	`:2` = the 2-letter code
`{country}` / `{country:2}`	`united-states` / `us`	
`{place}`	`springfield-il`	short for `{city}-{state:2}` (outside the US, `{city}-{country:2}`)

Folder pattern	Folder name
`{YYYY}-{MM} {title} ({place})` (the default)	`2019-04 Easter at grandma's (springfield-il)`
`{YYYY}-{MM} {title} ({street}-{city}-{state:2})`	`2019-04 Easter at grandma's (maple-street-springfield-il)`
`{YYYY}-{MM} ({city}-{state:2})`	`2019-04 (springfield-il)`
`{YYYY}-{MM}-{DD} {title}`	`2019-04-20 Easter at grandma's`
`{YYYY}-{MM} {title} ({city}, {state})`	`2019-04 Easter at grandma's (springfield, illinois)`
`{YYYY}-{MM} {title} ({city}-{country:2})`	`2019-07 Summer trip (paris-fr)`
a group in more than one place	`2019-04 Spring trip (springfield-il, chicago-il)`
a group with no position	`2019-04 Easter at grandma's`

Place tokens are lower case with `-` for spaces; empty ones are dropped with their separators, and empty brackets disappear. A location folder always sits in its year folder (`organized\2019\2019-04 Easter at grandma's (...)`), and its name starts with the month, so DA knows the photos are in the right place.

You can also make one by hand. Rename a month folder (or make a new folder next to it and move some of that month's photos in, with their `.meta.json` files), keeping the month at the start: `2019-04` -> `2019-04 Easter at grandma's`. `da validate` and `da run` leave photos there. A photo from another month in that folder is still moved back to its own month folder. New photos from `da run` go into the plain month folder.

If something goes wrong

What you see	What to do
'da' is not recognized	Close the terminal and open a **new** one. If that doesn't help, install DA again.
Windows protected your PC	Click **More info**, then **Run anyway**. DA isn't signed by a company, so Windows asks once.
Not inside an archive	Open the archive with its `Open DA.cmd`, or type `da` and pick one, or add `--archive D:\jane-archive-199x` to the command.
DA.exe is in use while updating	Close every DA window, then install again.
The system cannot find the path specified, or *path too long for Windows*	The file's path is 260 characters or more. Turn on long paths (run `da install` again, or the command there), open a new DA window and run again.
Sharing says *UnknownApiNameOrVersion: name: gmail version: v1*	An older DA. Update it (`da update` , or the newest DA.exe), then try again; the sign-in you made with `da share --setup` stays.
The dashboard says *Could not find a suitable TLS CA certificate bundle ... cacert.pem* (or another file is missing)	An older DA.exe lost files when the window that started the dashboard closed. Update DA (`da update`), then `da dashboard --stop` and `da dashboard` .
Looking up places (or homes) says *couldn't check the lookup service's security certificate* (*CERTIFICATE_VERIFY_FAILED*)	Waiting won't fix it. Update DA (`da update`): DA now checks certificates the way Windows does. Still there? An antivirus that checks secure connections is usually in the way: turn off its HTTPS or web-shield scanning, or allow `nominatim.openstreetmap.org` . On a work or school network, ask whoever looks after it. The photos keep their GPS; run `da location lookup` again once it's fixed.
Sharing says *Gmail didn't send it (403): Gmail API has not been used in project ...*	The Gmail API isn't turned on in your Google Cloud project: **APIs & Services** > **Library** > **Gmail API** > **Enable** (see Set up sharing), wait a minute, try again.
Which DA is this?	`da version` shows its version. In PowerShell, `Get-Command da` shows which one runs: `C:\Scripts\DA\DA.exe` , or `da.cmd` for DA installed with Python. In Explorer, **Properties > Details** of `DA.exe` shows its version too.
A run seems stuck	It's probably working through a big folder. Big runs take hours; the step number shows where it is. To stop, press **Ctrl+C**. Photos already filed stay filed, and running it again carries on with the rest.
A photo was filed with the wrong date	The camera's clock was wrong. Rename the file in `organized` with the right date at the start, and move it to the right month folder.
Nothing to file	Everything in that folder is either filed or waiting for you. See **3 What needs review**.

What you see	What to do
Files are in `_duplicates-holding` that you want back	**4 Undo last set-aside**, or `da intake undo "<run folder>\set-aside-moved.csv"`.
You want to know what DA did, and when	Every command is written to the archive's log, `logs\da.jsonl`: when it started, how it ended, on which computer, and what it changed (files moved, renamed, set aside, indexed, contact sheets and reports made), with the run folder and undo file for each. Errors are there with the full details, which helps when you ask for help.

Reference

How DA works out a date

In this order:

1. A date **and time** in the file name (`2024_07_04_10_15_00.jpg`, `20240704_101500.jpg`).
2. The date stored **inside** the file:
   - photos: the camera's "date taken", including iPhone HEIC photos. A camera whose clock was never set (1 January 1970) doesn't count.
   - PDFs: when the PDF was made (scanned, printed or saved).
   - Word, Excel, PowerPoint: when the document was created (the older `.doc .xls .ppt` too).
3. A date **without a time** in the name (`2010-04-27`, US-style `04-27-2010` or `1-11-09`, or written out: `January 4th 2009`). If the date inside the file is from that same day, its time is used too.
4. A **month and year** in the name (`Resume April-2010`): the date inside the file, if it's from that month. Otherwise only the month is known, which isn't enough, and the file stays for review.
5. A **year only** (`Beach 1998.jpg`): not enough; it stays for review.

A **scanned print** is dated differently: never by its own date (the day it was scanned), but by a date set by hand, a date typed into it before, its name, or the folders it came in from - and it needs a month. See Scanned prints. A camera listed in `cameras.json` with `shift +1h` (or `+1y` ...) has its dates moved by that much; with `none`, they aren't used.

When the name and the date inside disagree, the name wins, and the file is listed for review.

Photos are named `photo_YYYYMMDD_HHMMSS_xxxx.jpg` and documents `doc_YYMMDD_HHMMSS_xxxx.pdf`; the last four characters come from the file's fingerprint, so two photos taken in the same second still get different names.

Photo or document?

PDFs and Word, Excel and PowerPoint files are always documents - even in `intake\photos\`, where DA files them with the documents. A picture (`.jpg .png .tif .bmp`) could be a photo or a scan, so the intake folder decides: in

`docs\` it's a document; in `scans\` it's a scanned print (a photo dated by its folders or by hand; `scans: scanphoto`; an older archive may still say `scans: document`); anywhere else it's a photo. To decide for a whole run: `da run intake\old-scans --kind document`.

Which archive a command works on

1. The one you name: `--archive D:\jane-archive-199x` (before or after the command).
2. Otherwise, the archive the folder you name is in.
3. Otherwise, the archive you're in: the terminal's current folder, or one above it.

Chimes when a command finishes

Add `--feedback` to any command (before or after it) to hear a short beep when it starts and a two-tone chime when it ends, so you can do something else while a long one runs:

```
da run intake\scans --feedback
```

To chime after every long command without typing it, set `enabled: true` under `feedback:` in the archive's `da-archive.yaml`: then any command that took a minute or more chimes when it ends (not quick ones, not the menu). `--no-feedback` keeps one command quiet either way.

Commands

Command	What it does
`da`	The menu
`da run [folder]`	File what's in intake (or one folder of it)
`da status`	This archive at a glance
`da run --template <name> [folder]`	File only the documents a template matches, by what's printed on them
`da template new <name> --from <file>` / `edit` / `test <name> <files>` / `list`	Make, change and try document templates
`da report [--open] [--refresh]`	A dated one-page PDF about the archive, in its `output` folder
`da dashboard [--stop] [--foreground] [--port N] [--no-open]`	The archive in your browser, kept up to date; runs in the background until `--stop`
`da setup <folder> [--years 1990-1999 | 1999 | 199x]` / `da setup --here`	Create an archive, or turn a folder (or the one you're in) into one
`da history [<file>] [--hash ...] [--duplicates] [--everywhere] [--kind ...] [--since <date>] [--limit N] [--no-save]`	What DA changed in this archive, newest first; or one file's whole story and copies; or every held copy (saved as CSV + JSON)
`da archives [--open N]`	The archives on this computer
`da archives move <archive> <new place> [--full] [--remove-old]`	Move an archive to another drive or folder; its index, undo logs and the list follow
`da archives delete <archive> [--also <folder>] [--forget]`	Delete an archive, only when every file in it exists elsewhere (Recycle Bin)
`da guide [--pdf]`	Open this guide (or its PDF)
`da db build` / `da db stats` / `da db query "<SQL>"`	The index: refresh, counts by year, search
`da intake status` / `da intake undo <moved.csv>`	Intake progress; put duplicates back
`da validate <folder>`	Check a folder (nothing moves)
`da contact-sheets <folder> [--force]`	Make the contact sheets for every folder of photos in it
`da tag [files or folders] [tags] [--remove tag]`	Automatic tags, or add or remove your own
`da tag --cameras`	The cameras found; asks who used each and saves it
`da tag --owner "<camera>" <name>`	Save who used a camera
`da tag ocr <folder> [--tag-only | --tag-and-move] [--again]`	Read the text in pictures; tag `text` / `document`; move documents
`da share <folders> --to <list> [--note "..."] [--send]`	Email a simple contact sheet from your Gmail (`--setup` once, `--lists`)

Command	What it does
`da find --passwords [<folder>] [--move] [--include-weak]`	Find documents that hold passwords; move them to `_secure-holding`
`da ocr [<folders>] [--dates | --secrets] [--again]`	Note each document's dates, and Secrets yes/no, in its sidecar
`da contact-sheets <folder> --kind document [--layout 2x2] [--force]`	A sheet per year of documents (first pages, pages, Secrets, dates)
`da rename [<folder>] [--pattern "..."] [--folder "..."] [--kind photo|video|document] [--apply] [--save]`	Rename and move filed files by the name and folder patterns (menu N; a preview without --apply; undo with `da intake undo`)
`da findit tags <folder> -r <tag>`	Find files by tag
`da location lookup [folder] [--offline]`	Look up the town and state of GPS photos; adds place tags
`da import <folders> [--apply] [--exclude <folder>] [--no-ocr]` / `da import --plan [--apply]`	Copy in what no archive has yet from a drive or folder, sorted into docs, photos and scans; `--plan`: what imports.yaml gives this archive
`da imports [--init <drives>] [--open]`	The import plan's progress: a table, and imports.html beside the archives
`da backup` / `da backup --setup <folder>`	Back up every archive on this computer to the backup drive (menu **K**); `--setup` once
`da split [--to <archive>] [--years 2000-2004]`	Move files from other years to the archive for them
`da merge --from <other archive>`	Copy in what another copy of the archive has and this one doesn't
`da location group <month>`	Group a month's GPS photos (`2019-04` , or its folder) into location folders (menu **L**)
`da location rename <folder> "<title>"`	Give a location folder a new title
`da location set <photos> --place "<town, state>"`	Set a place by hand (`--same-as <photo>` , `--clear`)
`da facial scan / people / suggest / rename / forget`	Find the faces in photos, learn people from Picasa's names, suggest names (you confirm)
`da picasa [<folders>] [--apply]`	Bring Picasa's faces, names, captions, stars and albums into the sidecars
`da similar [<folders>] [--apply]`	Set aside resized copies of filed photos (the same picture, smaller); the largest stays
`da pair-movies [--apply]`	Put Live Photo videos and voice notes left in intake beside their filed photo
`da heic-to-jpg <folder> [--out <folder>]`	JPG copies of HEIC photos
`da date list` / `da date set <photos> --date 1998-07` / `da date shift <photos> --by +1h`	Fix dates (menu **D**)

Command	What it does
`da review [<folder>] [--guess] [--file]`	Photos waiting for a date in the review folder, with the best guess for each; `--file` files the dated ones (menu **V**)
`da cameras [--detect] [--open]` / `da cameras set "<camera>" [--owner] [--type] [--date] [--years]` / `da cameras add "<film camera>"`	Every camera and scanner (cameras.json): owner, type, how to read its dates, years (menu **C**)
`da run <folder> --date 1987-03`	Give every photo there with no sure date this date as it's filed (a box of prints)
`da sidecar upgrade [folder]`	Bring older sidecars up to date (menu **J**)
`da version`	Which version of DA this is
`da update [--check]`	Look for a newer DA and put it in place (DA.exe or Python)
`<DA.exe> install`	Install or update DA on this computer
